

**OPTIMASI JADWAL KERJA DAN ISTIRAHAT MENGGUNAKAN
ALGORITMA GENETIKA PENDEKATAN PADA PEKERJA DENGAN
POLA TIDAK TERATUR**

TUGAS AKHIR



Disusun Oleh :
BARETTA PANUJU LUHUR
123220204

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

OPTIMASI JADWAL KERJA DAN ISTIRAHAT MENGGUNAKAN ALGORITMA GENETIKA PENDEKATAN PADA PEKERJA DENGAN POLA TIDAK TERATUR

TUGAS AKHIR

Tugas Akhir ini sebagai salah satu syarat untuk memperoleh gelar sarjana Informatika
Universitas Pembangunan Nasional “Veteran” Yogyakarta



Disusun Oleh :
BARETTA PANUJU LUHUR
123220204

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN”
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

**OPTIMASI JADWAL KERJA DAN ISTIRAHAT MENGGUNAKAN
ALGORITMA GENETIKA PENDEKATAN PADA PEKERJA DENGAN
POLA TIDAK TERATUR**

Disusun oleh:
Baretta Panuju Luhur
123220204

Telah diperiksa dan disetujui oleh pembimbing untuk diseminarkan
Pada tanggal:

Menyetujui,
Pembimbing

Dr. Heru Cahya Rustamaji, S.Si., M.T.
NIDN. 0514067101

SURAT PERNYATAAN KARYA ASLI TUGAS AKHIR

Sebagai mahasiswa Program Studi Informatikai Fakultas Teknik Industri Universitas Pembangunan Nasional “Veteran” Yogyakarta, yang bertanda tangan di bawah ini, saya:

Nama : BARETTA PANUJU LUHUR
NIM : 123220204

Menyatakan bahwa karya ilmiah saya yang berjudul:

Optimasi Jadwal Kerja Dan Istirahat Menggunakan Algoritma Genetika Pendekatan Pada Pekerja Dengan Pola Tidak Teratur

Merupakan karya asli saya dan belum pernah dipublikasikan dimanapun. Apabila di kemudian hari, karya saya disinyalir bukan merupakan karya asli saya, maka saya bersedia menerima konsekuensi apa pun yang diberikan Program Studi Informatika Fakultas Teknik Industri Universitas Pembangunan Nasional “Veteran” Yogyakarta kepada saya.

Demikian surat pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Yogyakarta
Pada Tanggal :

Yang Menyatakan

Baretta Panuju Luhur
NIM. 123220204

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini:

Nama : BARETTA PANUJU LUHUR
NIM : 123220204
Prodi : Informatika

Dengan ini saya menyatakan bahwa judul Tugas Akhir

Optimasi Jadwal Kerja Dan Istirahat Menggunakan Algoritma Genetika Pendekatan Pada Pekerja Dengan Pola Tidak Teratur

adalah hasil kerja saya dan benar bebas dari plagiasi kecuali cuplikan serta ringkasan yang terdapat di dalamnya telah saya jelaskan sumbernya (Sitasi) dengan jelas. Apabila pernyataan ini terbukti tidak benar maka saya bersedia menerima sanksi sesuai peraturan Mendiknas RI No 17 Tahun 2010 dan Peraturan Perundang-undangan yang berlaku.

Demikian surat pernyataan ini saya buat dengan penuh tanggung jawab.

Dibuat di : Yogyakarta
Pada Tanggal :

Yang Menyatakan

Baretta Panuju Luhur
NIM. 123220204

ABSTRAK

Abstrak memuat tiga bagian utama yaitu masalah penelitian, metode penelitian, dan hasil penelitian yang dituliskan dalam tiga paragraf. Paragraf pertama, mencakup permasalahan yang diangkat menjadi topik penelitian. Paragraf kedua, berisi metode penelitian yang digunakan, beserta proses akuisisi data, serta metode pengembangan sistem yang digunakan. Paragraf ketiga, memaparkan hasil penelitian, termasuk hasil pengujian, jawaban dari permasalahan penelitian, serta kontribusi penelitian.

Abstrak ditulis dengan menggunakan bahasa Indonesia yang baku dengan jumlah kata minimal 300 kata dan tidak lebih dari satu halaman. Abstrak harus mampu menggambarkan keseluruhan dari penelitian yang telah dilaksanakan, mulai dari masalah penelitian sampai hasil penelitian. Tidak diperbolehkan mengacu pustaka, gambar, maupun tabel.

Pada akhir abstrak, dicantumkan kata kunci yang merupakan kata-kata atau frasa penting yang mencerminkan inti dari topik penelitian. Kata kunci diambil dari bidang penelitian dan topik penelitian.

Kata Kunci: Jumlah kata kunci antara 3 sampai dengan 5 kata dipisah dengan koma

ABSTRACT

The abstract consists of three main parts containing research problems, research methods, and research results, written in three paragraphs. The first paragraph, includes problems raised as a research topic. Abstract in English is italicized.

Keywords: xxxxxxxx, xxxx, xxxxxxxxxx, xxxxxxxxxxxxxx

KATA PENGANTAR

DAFTAR ISI

SURAT PERNYATAAN.....	iv
KARYA ASLI TUGAS AKHIR.....	iv
PERNYATAAN BEBAS PLAGIASI.....	v
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Masalah.....	1
1.2 Rumusan Masalah.....	2
1.3 Batasan Masalah.....	3
1.4 Tujuan Penelitian.....	3
1.5 Manfaat Penelitian.....	3
1.6 Tahapan Penelitian.....	4
1.7 Sistematika Penulisan.....	4
BAB II TINJAUAN LITERATUR.....	7
2.1 Landasan Teori.....	7
2.1.1 Model Dua Proses Regulasi Tidur (Two-Process Model)	7
2.1.2 Social Jetlag	8
2.1.3 Sleep Debt dan Fatigue Index	9
2.1.4 Sleep Regularity Index (SRI)	10
2.1.5 Optimasi Kombinatorial dan Penjadwalan	10
2.1.6 Genetic Algorithm (Algoritma Genetika)	11
2.1.7 Hill Climbing	12
2.1.8 Genetic Algorithm dan Hill Climbing	13
2.1.9 Fungsi Fitness Multikriteria	14
2.1.10 Penanganan Kendala (Constraint Handling)	14
2.1.11 Baseline dalam Evaluasi Algoritma	15
2.1.12 Representasi Waktu dan Aktivitas	15
2.1.13 Fungsi Fitness dan Evaluasi Solusi	17
2.1.14 Penanganan Kendala	17
2.1.15 Parameter Algoritma	17
2.2 Tinjauan Sumber Pustaka.....	18
2.3 Gap Penelitian dan Posisi Penelitian.....	20
BAB III METODOLOGI PENELITIAN.....	22
3.1 Jenis dan Rancangan Penelitian.....	22
3.2 Data Penelitian.....	22
3.3 Alur Penelitian.....	23

3.4 Pra-Pemrosesan Data.....	25
3.4.1 Parsing Slot Aktivitas	25
3.4.2 Pembentukan Mandatory Core	25
3.4.3 Pembentukan Transit Slot	26
3.4.4 Pembentukan Mandatory Absolute	26
3.4.5 Feasibility Check	26
3.5 Prosedur Eksekusi Genetic Algorithm dan Hill Climbing.....	27
3.5.1 Inisialisasi Populasi	28
3.5.2 Evaluasi Fitness	29
3.5.3 Elitisme dan Seleksi Parent	30
3.5.4 Crossover	30
3.5.5 Mutasi	31
3.5.6 Penyempurnaan Solusi Menggunakan Hill Climbing	32
3.5.7 Hasil Eksekusi Genetic Algorithm dan Hill Climbing	32
3.6 Optimasi Jadwal Tidur Mingguan.....	33
3.6.1 Optimasi Hari Pertama	33
3.6.2 Carry-Over Kondisi Antarhari	34
3.6.3 Optimasi Hari Kedua hingga Hari Ketujuh	34
3.6.4 Penyusunan Jadwal Mingguan	34
3.6.5 Hasil Optimasi Mingguan	35
3.7 Konfigurasi Parameter Eksperimen Utama.....	35
3.8 Baseline Comparison.....	35
3.8.1 Greedy Schedule	35
3.8.2 Fixed Schedule	36
3.8.3 Evaluasi Komparatif	36
3.9 Evaluasi Sistem.....	37
3.9.1 Weekly Optimization Test	37
3.9.2 Baseline Comparison	24
3.9.3 Multi-Subject Test	38
3.9.4 Indikator Evaluasi	39
3.9.5 Analisis Hasil Pengujian	39
3.10 Implementasi Program.....	39
3.10.1 Arsitektur Program	40

3.10.2 Modul Pembacaan dan Pra-Pemrosesan Data	40
3.10.3 Modul Optimasi Harian	40
3.10.4 Modul Optimasi Mingguan	41
3.10.5 Modul Evaluasi dan Baseline Comparison	41
3.10.6 Modul Antarmuka Streamlit	41
3.10.7 Struktur Implementasi Program	42
3.10.8 Hasil Implementasi	42
BAB IV HASIL PENGUJIAN DAN PEMBAHASAN.....	43
4.1 Implementasi Sistem.....	43
4.1.1 Implementasi Dataset dan Normalisasi	43
4.1.2 Representasi Kromosom dan Constraint	43
4.1.3 Implementasi Fungsi Fitness	44
4.1.4 Implementasi Genetic Algorithm	44
4.1.5 Implementasi Hill Climbing	44
4.1.6 Implementasi Optimasi Mingguan dan API	45
4.1.7 Implementasi Antarmuka dan Keluaran	45
4.1.8 Pseudocode dan Cuplikan Kode Inti	46
4.2 Hasil Kalibrasi Parameter.....	49
4.3 Hasil Pengujian Optimasi Mingguan.....	50
4.4 Hasil Demo Pipeline Skripsi.....	53
4.5 Perbandingan dengan Baseline.....	54
4.6 Pembahasan Hasil.....	55
4.6.1 Analisis Ringkas Per Subjek	55
4.7 Ringkasan Pembahasan.....	56
4.8 Keterbatasan Pengujian.....	57
BAB V PENUTUP.....	58
5.1 Kesimpulan.....	58
5.2 Keterbatasan Penelitian.....	58
5.3 Saran.....	59
5.4 Penutup.....	59

DAFTAR TABEL

Tabel 2. 1 State of The Art.....	25
Tabel 3. 1 Kode Aktivitas pada Representasi Kromosom.....	15
Tabel 3. 2 Komponen Dynamic Penalty pada Fungsi Fitness.....	20
Tabel 3. 3 Modul Utama Implementasi Program.....	
Tabel 4.1 Persamaan Fitness yang Diterapkan dalam Implementasi.....	44
Tabel 4.2 Hasil Kalibrasi Parameter Menggunakan Grid Search.....	49
Tabel 4.3 Statistik Optimasi Mingguan Seluruh Subjek.....	50
Tabel 4.4 Statistik dan Hasil Berpasangan Metode pada Hari Rabu.....	54
Tabel 4.5 Rincian Perhitungan Komponen Fitness pada Kasus Ekstrem.....	55

DAFTAR GAMBAR

Gambar 2. 1 Two Process Model of Sleep Regulation.....	7
Gambar 2. 2 Pengukuran Social Jetlag.....	8
Gambar 2. 3 Genetic Algorithm dan Hill Climbing.....	
Gambar 3. 1 Alur Metodologi Penelitian.....	29
Gambar 3. 2 Representasi 48 Slot Harian.....	15
Gambar 3. 3 Representasi Kromosom Jadwal Harian.....	16
Gambar 3. 4 Mandatory Core, Transit Slot, dan Mandatory Absolute.....	31
Gambar 3. 5 Alur Repair Chromosome.....	23
Gambar 3. 6 Alur Genetic Algorithm.....	34
Gambar 3. 8 Alur Genetic Algorithm dan Hill Climbing.....	
Gambar 3. 10 Grid Search.....	40
Gambar 3. 11 Baseline Comparison dan Evaluasi.....	42
Gambar 4.1 Alur implementasi SORA Academic.....	46
Gambar 4.2 Rerata, median, dan hasil berpasangan fitness menurut hari.....	53

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Tidur merupakan kebutuhan fisiologis mendasar yang memengaruhi hampir seluruh aspek kesehatan dan fungsi kognitif manusia. Bukti ilmiah yang terus berkembang menunjukkan bahwa bukan hanya durasi tidur yang cukup, melainkan juga regularitas atau konsistensi pola tidur dari hari ke hari merupakan faktor yang sama pentingnya bagi kesehatan jangka panjang. (Borbély 1982) melalui *Two-Process Model* menegaskan bahwa regulasi tidur dikendalikan oleh interaksi dua proses, yakni proses homeostat (tekanan tidur yang terakumulasi saat terjaga) dan proses sirkadian (ritme biologis 24 jam), sehingga ketidaksesuaian antara keduanya akan mengakibatkan degradasi kualitas tidur secara sistematis. (Dijk and Czeisler 1995) lebih lanjut membuktikan bahwa pacemaker sirkadian internal manusia secara aktif menentukan propensitas tidur, struktur tidur, dan aktivitas gelombang lambat EEG, menegaskan pentingnya keselarasan jadwal tidur dengan ritme biologis.

Permasalahan yang kini meluas di masyarakat modern adalah ketidaksesuaian antara jam biologis internal dan tuntutan jadwal sosial-aktivitas eksternal, yang dikenal sebagai social jetlag (Wittmann et al. 2006). (Roenneberg et al. 2012) mendefinisikan social jetlag sebagai selisih waktu tidur antara hari kerja dan hari libur yang mencerminkan seberapa jauh seseorang hidup di luar ritme sirkadiannya akibat tuntutan jadwal sosial, dan menemukan bahwa kondisi ini berkorelasi signifikan dengan peningkatan indeks massa tubuh serta penurunan kesehatan metabolik. (Wittmann et al. 2006) menambahkan bahwa social jetlag yang persisten berdampak pada peningkatan konsumsi stimulan, penurunan performa kognitif, dan peningkatan kelelahan kumulatif. Di kalangan mahasiswa dan pekerja muda Indonesia, permasalahan ini semakin relevan seiring intensifikasi penggunaan perangkat digital dan ketidakteraturan jadwal studi maupun kerja.

Urgensi penanganan regularitas tidur diperkuat oleh serangkaian studi terbaru. (Phillips et al. 2017) memperkenalkan *Sleep Regularity Index* (SRI) sebagai metrik kuantitatif yang mengukur konsistensi jadwal tidur-bangun, dan menunjukkan bahwa pola tidur yang tidak konsisten berkorelasi signifikan dengan penurunan performa akademik serta keterlambatan fase sirkadian. (Windred et al. 2024) lebih jauh membuktikan bahwa SRI merupakan prediktor mortalitas yang lebih kuat dibandingkan durasi tidur semata, sementara (Sletten et al. 2023) dalam sebuah konsensus internasional menegaskan bahwa regularitas tidur merupakan komponen penting dari higiene tidur yang sehat. (Huang, Mariani, and Redline 2020) juga menemukan bahwa iregularitas tidur berhubungan dengan peningkatan risiko kejadian kardiovaskular dalam studi kohort besar MESA, semakin memperkuat dasar ilmiah perlunya intervensi berbasis regularitas tidur.

Dari perspektif ilmu komputer, permasalahan penjadwalan tidur dan aktivitas dapat diformulasikan sebagai masalah optimasi kombinatorial berkendala. Dalam formulasi ini, satu hari dibagi menjadi 48 slot diskret dengan interval 30 menit, dan solusi yang dihasilkan harus memenuhi kendala aktivitas wajib sekaligus mempertimbangkan kualitas pemulihan tidur, durasi tidur aktual, tingkat kelelahan akibat aktivitas, serta regularitas waktu mulai tidur antarhari. Regularitas tidur dalam penelitian ini tidak dihitung menggunakan nilai *Sleep Regularity Index* (SRI) secara penuh, melainkan dimodelkan melalui penalti terhadap perubahan waktu mulai tidur utama dibandingkan hari sebelumnya. Permasalahan ini memiliki ruang solusi yang besar sehingga pendekatan heuristik sederhana, seperti *greedy schedule*, berpotensi menghasilkan solusi suboptimal karena tidak mengevaluasi konsekuensi jadwal secara menyeluruh.

Di sisi lain, Goldberg (Goldberg 1989) membuktikan bahwa *Genetic Algorithm* (GA) merupakan pendekatan metaheuristik berbasis populasi yang efektif untuk ruang solusi besar dan tidak linier melalui mekanisme seleksi, pindah silang, dan mutasi. Meskipun demikian, GA murni rentan terhadap konvergensi prematur dan memiliki kelemahan inheren pada fase eksploitasi lokal (*local exploitation*). GA sering kali membutuhkan waktu komputasi (konvergensi) yang sangat lama untuk memperbaiki detail-detail kecil pada kromosom, seperti menutup celah waktu kosong (*micro-gaps*) atau merapikan waktu transisi antar aktivitas. Untuk menutupi kelemahan tersebut, (Moscato 1989) kemudian memperkenalkan *Genetic Algorithm dan Hill Climbing* (MA) yang menggabungkan eksplorasi global GA dengan eksploitasi lokal *Hill Climbing* (HC), menghasilkan konvergensi lebih cepat dan kualitas solusi lebih tinggi pada masalah optimasi kombinatorial berkendala.

Penelitian ini memperkenalkan sistem sebagai implementasi Genetic Algorithm dan Hill Climbing yang menggabungkan Genetic Algorithm dan Hill Climbing untuk mengoptimalkan jadwal tidur dan aktivitas selama tujuh hari. Komponen inti sistem mencakup mekanisme *repair constraint* untuk menjaga kelayakan jadwal, fungsi fitness komposit yang mempertimbangkan *recovery reward*, durasi tidur aktual, *dynamic penalty*, dan penalti regularitas waktu tidur antarhari, serta modul evaluasi multi-subjek yang menghasilkan laporan terstruktur. Sistem diimplementasikan melalui antarmuka web berbasis Streamlit untuk mendukung interaksi pengguna dan visualisasi hasil optimasi. Penelitian ini berupaya memberikan pendekatan komputasional terhadap perencanaan istirahat pada pola aktivitas yang tidak teratur melalui metode metaheuristik hibrida.

1.2 Rumusan Masalah

Bagaimana merancang dan mengimplementasikan sistem optimasi jadwal tidur dan aktivitas selama tujuh hari berbasis *Genetic Algorithm* dan *Hill Climbing* yang menggabungkan *Genetic Algorithm* dan *Hill Climbing*, menerapkan mekanisme repair constraint pada representasi 48 slot waktu per hari, menggunakan fungsi fitness yang mempertimbangkan pemulihan tidur, kelelahan akibat aktivitas, durasi tidur, dan penalti regularitas waktu tidur antarhari, serta membandingkan kinerjanya terhadap metode *greedy schedule* dan *fixed schedule* pada pengujian multi-subjek melalui aplikasi berbasis web?

1.3 Batasan Masalah

Agar penelitian ini dapat diselesaikan secara fokus dan terukur, batasan-batasan masalah yang ditetapkan adalah sebagai berikut:

1. Sistem hanya mengoptimalkan jadwal tidur dalam cakrawala waktu tujuh hari ke depan; optimasi jangka panjang lebih dari tujuh hari berada di luar cakupan penelitian.
2. Evaluasi komparatif dilakukan terhadap dua metode *baseline*, yaitu *greedy* dan *fixed schedule*; algoritma metaheuristik lain tidak diuji dalam penelitian ini.
3. Parameter algoritma (ukuran populasi, laju mutasi, laju crossover, jumlah iterasi Hill Climbing) dioptimalkan melalui eksperimen kalibrasi, bukan melalui prosedur auto-tuning adaptif.
4. Penelitian ini hanya berfokus pada aspek teknis pemodelan dan evaluasi kualitas jadwal tidur, tanpa membahas aspek klinis diagnostik medis terhadap gangguan tidur.
5. Aktivitas yang dimodelkan terdiri dari tidur utama, kerja, nap, santai, dan kuliah.
6. Regularitas tidur dihitung sebagai penalti perubahan waktu mulai tidur utama, bukan perhitungan SRI klinis penuh.
7. Data biometrik wearable tidak digunakan.
8. Sistem menghasilkan rekomendasi komputasional dan bukan alat diagnosis medis.

1.4 Tujuan Penelitian

Merancang dan mengimplementasikan sistem *Sleep Optimization and Rest Arrangement* (SORA) berbasis *Genetic Algorithm* dan *Hill Climbing* yang menggabungkan *Genetic Algorithm* dan *Hill Climbing*, dilengkapi mekanisme *repair constraint* pada representasi 48 slot waktu per hari serta fungsi fitness komposit yang mempertimbangkan *recovery reward*, durasi tidur aktual, *dynamic penalty*, dan penalti regularitas waktu tidur antahari. Penelitian ini juga bertujuan mengevaluasi kinerja sistem secara komparatif terhadap metode *greedy schedule* dan *fixed schedule* pada skenario optimasi mingguan dan pengujian multi-subjek melalui aplikasi web Streamlit.

1.5 Manfaat Penelitian

Secara akademis, penelitian ini diharapkan dapat memperkaya kajian mengenai penerapan algoritma metaheuristik hibrida pada permasalahan optimasi jadwal tidur dan aktivitas, khususnya melalui penggunaan penalti regularitas waktu tidur antahari yang terinspirasi oleh konsep *Sleep Regularity Index*. Secara praktis, penelitian ini menghasilkan purwarupa aplikasi yang dapat membantu pengguna merencanakan waktu tidur dan aktivitas berdasarkan kendala jadwal yang dimiliki. Bagi institusi, sistem yang dibangun dapat menjadi referensi tambahan dalam kajian optimasi, kecerdasan buatan, dan pengembangan aplikasi pendukung perencanaan aktivitas personal.

1.6 Tahapan Penelitian

Penelitian ini menggunakan pendekatan rancangan dan implementasi algoritma optimasi. Pengembangan aplikasi dilakukan dengan model pengembangan perangkat lunak Prototyping, sedangkan evaluasi kinerja algoritma dilakukan melalui eksperimen komputasional. Eksperimen tersebut mencakup kalibrasi parameter menggunakan grid search, optimasi jadwal mingguan, perbandingan terhadap metode baseline, dan pengujian multi-subjek. Adapun tahapan utama penelitian ini meliputi:

1. Perencanaan dan Studi Literatur

Mengkaji literatur terkait regulasi tidur sirkadian, metrik kualitas tidur (*sleep debt*, *fatigue*, *SRI*), serta *Genetic Algorithm* dan *Hill Climbing*.

2. Akuisisi Data Preferensi Subjek

Mengumpulkan informasi jendela waktu tidur yang diizinkan, target durasi tidur, dan kendala aktivitas harian.

3. Perancangan Sistem Optimasi

Merancang representasi solusi berbasis 48 slot per hari, formulasi *fitness function* multikriteria, dan struktur *Genetic Algorithm* dan *Hill Climbing*.

4. Implementasi Sistem

Implementasi menggunakan bahasa pemrograman Python dengan pustaka inti optimasi (src/sora), layanan API berbasis FastAPI (api.py), serta antarmuka web berbasis React dan Streamlit.

5. Evaluasi dan Analisis Hasil

Membandingkan *Genetic Algorithm* dan *Hill Climbing* terhadap *baseline* (*greedy* dan *fixed schedule*) menggunakan metrik fitness, fatigue akhir, dan durasi tidur.

1.7 Sistematika Penulisan

adapun proses penulisan skripsi terdapat lima bagian yaitu:

BAB I : PENDAHULUAN

Berisi latar belakang masalah, rumusan masalah, batasan masalah, tujuan dan manfaat penelitian, serta sistematika penulisan.

BAB II : TINJAUAN LITERATUR

Membahas dasar teori dan konsep yang relevan, mencakup kajian mengenai *scheduling optimization*, arsitektur Algoritma Genetika, serta teori-teori terkait Ritme Sirkadian dan kebutuhan tidur.

BAB III : METODOLOGI PENELITIAN

Menjelaskan pendekatan *Prototyping*, tahapan pengumpulan data, perancangan teknis model Algoritma Genetika (representasi kromosom dan operator), serta formulasi matematis dari fungsi *fitness*.

BAB IV : HASIL DAN PEMBAHASAN

Memaparkan hasil implementasi dan pengujian model. Fokus utama adalah pada hasil studi komparasi *Genetic Algorithm* dan *Hill Climbing* vs. *Baseline* dan analisis kinerja berdasarkan metrik pemulihan fisiologis.

BAB V : PENUTUP

Berisi kesimpulan yang ditarik berdasarkan hasil pembahasan dan saran serta rekomendasi untuk pengembangan penelitian lanjutan.

BAB II

TINJAUAN LITERATUR

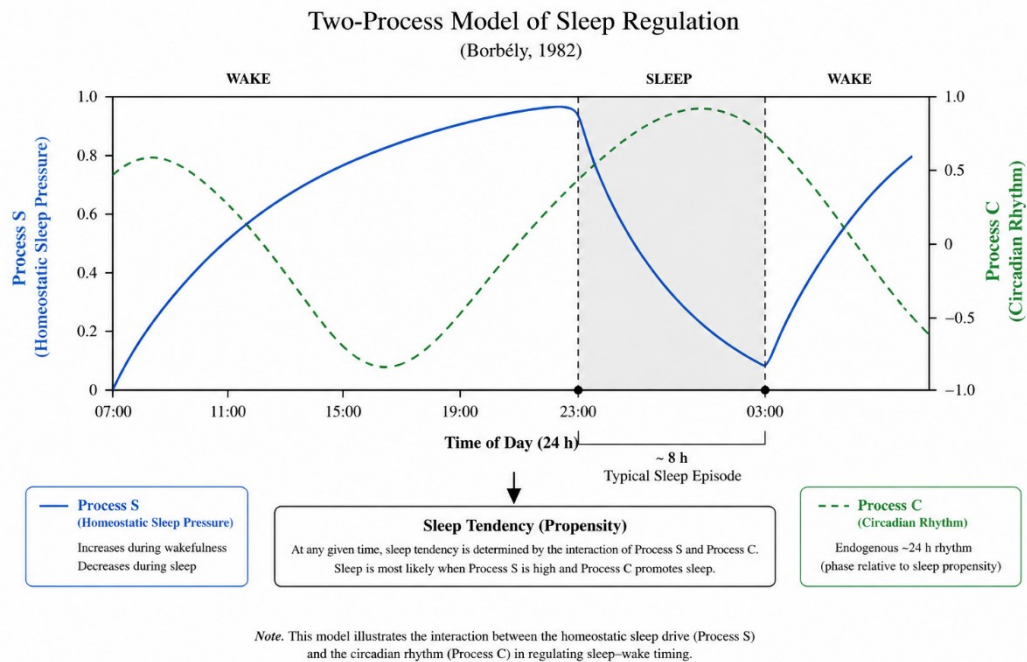
2.1 Landasan Teori

Subbab ini menyajikan dasar teori dan konsep yang menjadi landasan penelitian. Teori-teori diuraikan secara hierarkis, dimulai dari konsep biologis yang melatarbelakangi permasalahan tidur, dilanjutkan dengan konstruk pengukuran yang relevan, kemudian fondasi matematis optimasi kombinatorial, dan diakhiri dengan teori algoritma yang diterapkan dalam sistem yang dikembangkan.

2.1.1 Model Dua Proses Regulasi Tidur (*Two-Process Model*)

Tidur diregulasi oleh dua proses fisiologis yang saling berinteraksi, sebagaimana dirumuskan oleh (Borbély 1982). Proses pertama, yang disebut proses *homeostatis* (Process S), merepresentasikan tekanan tidur yang terakumulasi selama seseorang terjaga dan berkurang secara eksponensial selama tidur berlangsung. Proses kedua, yang disebut proses sirkadian (Process C), mencerminkan osilasi biologis endogen dengan periode mendekati 24 jam yang dikendalikan oleh nukleus suprakiasmatik (SCN) di hipotalamus. Propensitas tidur pada setiap titik waktu ditentukan oleh interaksi aditif kedua proses ini, di mana tidur yang optimal cenderung terjadi ketika Process S mencapai ambang atas bersamaan dengan fase sirkadian yang selaras.

Dijk dan Czeisler (Dijk and Czeisler 1995) memperkuat model ini melalui paradigma desinkronisasi paksa (*forced desynchrony protocol*) yang memisahkan siklus tidur-bangun dari ritme sirkadian internal. Eksperimen tersebut membuktikan bahwa pacemaker sirkadian internal secara aktif menentukan distribusi tidur REM, proporsi gelombang lambat EEG, dan efisiensi tidur. Temuan ini menegaskan bahwa kualitas tidur tidak hanya bergantung pada durasi, tetapi juga pada keselarasan temporal antara jadwal tidur dan fase sirkadian. Variabilitas individual pada laju akumulasi dan peluruhan Process S (Achermann 2003) menjadikan pemodelan tidur perlu mempertimbangkan parameter yang dipersonalisasi.



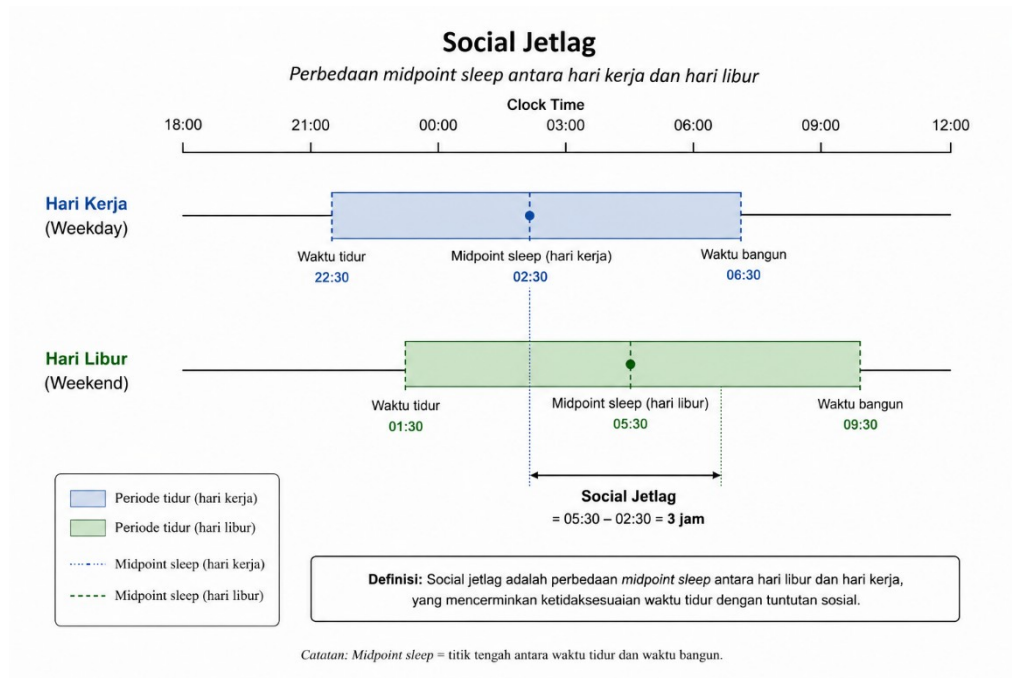
Gambar 2. 1 Two Process Model of Sleep Regulation

2.1.2 Social Jetlag

Social jetlag adalah kondisi ketidakselarasan antara jam biologis internal individu dengan jadwal sosial yang dituntut oleh lingkungan eksternal, seperti jam kerja atau jam perkuliahan. Istilah ini diperkenalkan oleh Wittmann et al. (Wittmann et al. 2006) berdasarkan survei kronobiologi berskala besar, dan dikembangkan lebih lanjut oleh Roenneberg et al. (Roenneberg et al. 2012). Kondisi ini diukur sebagai selisih waktu titik tengah tidur antara hari kerja dan hari libur. Individu dengan social jetlag yang substansial secara fisiologis mengalami beban yang sebanding dengan perjalanan melintasi zona waktu secara berulang, yang memberikan tekanan kumulatif pada sistem sirkadian.

Wittmann et al. (Wittmann et al. 2006) menemukan bahwa social jetlag berkorelasi positif dengan konsumsi stimulan seperti kafein dan nikotin, serta dengan peningkatan kelelahan yang dilaporkan secara subjektif. Roenneberg et al. (Roenneberg et al. 2012) melaporkan bahwa setiap satu jam social jetlag berhubungan dengan peningkatan sekitar 33% odds ratio kelebihan berat badan dalam sampel yang diteliti, yang mencerminkan dampak gangguan sirkadian terhadap regulasi metabolik, meskipun sifat kausalitasnya masih menjadi subjek penelitian lebih lanjut.

Dalam penelitian ini, konsep social jetlag menjadi dasar penting dalam menilai bahwa jadwal tidur yang baik tidak cukup hanya berdurasi memadai, tetapi juga perlu menjaga kestabilan waktu tidur antahari.



Gambar 2. 2 Pengukuran Social Jetlag.

2.1.3 Sleep Debt dan Fatigue Index

Sleep debt didefinisikan sebagai defisit kumulatif antara durasi tidur aktual dan kebutuhan tidur biologis individu (Van Dongen et al. 2003). Pemulihan dari sleep debt menunjukkan dinamika yang tidak linear: defisit jangka pendek memerlukan beberapa malam tidur adekuat untuk pulih, sedangkan defisit kronik memerlukan periode pemulihan yang lebih panjang (Belenky et al. 2003). Van Dongen et al. (Van Dongen et al. 2003) membuktikan melalui desain dosis-respons bahwa restriksi tidur kronik pada 6 jam per malam selama 14 hari menghasilkan penurunan kognitif yang secara kuantitatif setara dengan dua malam deprivation total, meskipun subjek tidak melaporkan kantuk berat akibat adaptasi perseptual.

Fatigue index merupakan konstruk yang lebih luas dari sleep debt, yang mencakup efek kelelahan bergantung waktu dalam siklus sirkadian serta beban aktivitas selama periode terjaga (Hursh et al. 2004). Model berbasis biomaterika seperti SAFTE (*Sleep, Activity, Fatigue, and Task Effectiveness*) mengkuantifikasi hubungan antara akumulasi kelelahan, fase sirkadian, dan efektivitas tugas (Jewett and Kronauer 1999). Dalam pemodelan penjadwalan, fatigue index berfungsi sebagai penalti terhadap jadwal yang mendistribusikan tidur pada fase sirkadian yang tidak kondusif, bahkan apabila durasi total tidur telah terpenuhi.

Dengan demikian, sleep debt dan fatigue index digunakan sebagai landasan teoritis untuk merepresentasikan kebutuhan pemulihan dan perubahan kelelahan akibat aktivitas. Dalam implementasi sistem yang dikembangkan, sleep debt tidak digunakan sebagai komponen fitness ternormalisasi yang berdiri sendiri, melainkan digunakan sebagai dasar penyesuaian target kebutuhan tidur apabila informasi riwayat tidur tersedia. Evaluasi

kandidat jadwal dilakukan melalui *recovery reward*, durasi tidur aktual, *dynamic penalty*, dan penalti regularitas waktu tidur antarhari.

2.1.4 Sleep Regularity Index (SRI)

Sleep Regularity Index (SRI) adalah metrik kuantitatif yang mengukur konsistensi pola tidur-bangun dari hari ke hari, yang diperkenalkan oleh Phillips et al. (Phillips et al. 2017) SRI mengukur probabilitas bahwa seseorang berada pada status tidur yang sama — tidur atau terjaga — pada dua titik waktu yang dipisahkan 24 jam. Secara matematis, SRI didefinisikan pada Persamaan 2.1.

$$SRI = -100 + (200 / T) \times \sum \delta(s(t), s(t + 24h)) \quad (2.1)$$

Keterangan:

T : jumlah total pasangan titik waktu yang dibandingkan dalam rentang pengukuran

$s(t)$: status tidur pada waktu t (bernilai 1 jika tidur, 0 jika terjaga)

δ : fungsi Kronecker delta yang bernilai 1 apabila kedua status identik, 0 apabila berbeda

Nilai SRI berkisar dari -100 , yang menunjukkan pola tidur-bangun yang sepenuhnya terbalik setiap harinya, hingga 100 , yang menunjukkan pola yang identik pada setiap hari pengukuran. Windred et al. (Windred et al. 2024) melaporkan dalam studi kohort UK Biobank bahwa SRI yang rendah berhubungan dengan risiko mortalitas yang lebih kuat dibandingkan durasi tidur yang tidak adekuat. Huang et al. (Huang, Mariani, and Redline 2020) menemukan dalam studi MESA bahwa irregularitas tidur berhubungan dengan peningkatan risiko kejadian kardiovaskular mayor secara prospektif. Konsep SRI digunakan sebagai landasan teoritis untuk merancang penalti regularitas tidur. Implementasi sistem tidak menghitung nilai SRI penuh, tetapi mengevaluasi perubahan waktu mulai tidur utama antarhari.

2.1.5 Optimasi Kombinatorial dan Penjadwalan

Optimasi kombinatorial adalah cabang matematika terapan yang berkaitan dengan pencarian solusi optimal dari ruang keputusan diskret yang berhingga namun sangat besar (Papadimitriou and Steiglitz 1982). Masalah penjadwalan (scheduling) merupakan salah satu kelas masalah optimasi kombinatorial yang paling banyak dikaji, di mana sumber daya dialokasikan terhadap slot waktu diskret dengan memenuhi seperangkat kendala sambil mengoptimalkan satu atau beberapa fungsi tujuan (Pinedo 2016). Kompleksitas komputasional sebagian besar varian masalah penjadwalan termasuk dalam kelas NP-hard dalam artian bahwa tidak diketahui algoritma deterministik yang mampu menyelesaikannya secara optimal dalam waktu polinomial untuk kasus umum (Garey and Johnson 2002).

Dalam konteks penjadwalan tidur, setiap slot waktu dapat dikodekan sebagai variabel keputusan biner, sehingga masalah ini bersifat alami sebagai binary scheduling problem. Ruang solusi yang terbentuk bersifat eksponensial terhadap jumlah slot dan cakrawala

perencanaan, sehingga pencarian secara exhaustive tidak layak secara komputasional. Kondisi ini menjadi justifikasi utama penggunaan algoritma metaheuristik yang mampu menjelajahi ruang solusi luas secara efisien tanpa menjamin optimalitas global, namun menghasilkan solusi berkualitas tinggi dalam waktu komputasi yang terbatas (Blum and Roli 2003).

Kendala dalam masalah penjadwalan umumnya dibedakan menjadi dua jenis (Pinedo 2016). Kendala keras (hard constraints) adalah batasan yang tidak boleh dilanggar dan secara langsung menentukan kelayakan (feasibility) suatu solusi. Kendala lunak (soft constraints) adalah preferensi yang diupayakan dipenuhi semaksimal mungkin, dan pelanggarananya mengakibatkan penurunan nilai tujuan melalui fungsi penalti.

$$x^* = \arg \max_{x \in \Omega} f(x) \quad (2.2)$$

Keterangan:

x^* = solusi optimal;
 x = kandidat solusi;
 Ω = ruang solusi;
 $f(x)$ = fungsi objektif atau fungsi fitness.

2.1.6 Genetic Algorithm (Algoritma Genetika)

Genetic Algorithm (GA) adalah metode pencarian dan optimasi berbasis populasi yang terinspirasi dari mekanisme seleksi alam dan evolusi genetis, yang diperkenalkan oleh (Holland 1975) dan dipopulerkan oleh Goldberg (Goldberg 1989). GA beroperasi pada populasi solusi kandidat yang disebut individu atau kromosom, yang berevolusi melalui sejumlah iterasi (generasi) menggunakan tiga operator utama: seleksi, pindah silang (crossover), dan mutasi.

Kromosom merepresentasikan satu solusi kandidat dalam ruang pencarian. Representasi kromosom merupakan keputusan desain kritis karena menentukan bagaimana ruang solusi dipetakan ke struktur data yang dapat dimanipulasi operator genetika (Rothlauf 2006). Representasi yang umum meliputi string biner, integer, real-valued, dan permutasi.

$$Fitness = f(kromosom) \quad (2.3)$$

Keterangan:

$Fitness$ = nilai kualitas individu;
 $kromosom$ = representasi solusi kandidat;
 f = fungsi evaluasi.

Seleksi menentukan individu mana yang akan berkontribusi pada generasi berikutnya. Skema yang lazim digunakan antara lain seleksi roda roulette yang memberi probabilitas seleksi proporsional terhadap fitness, seleksi turnamen yang memilih individu terbaik dari subset populasi acak, dan elitisme yang menjamin individu terbaik selalu

dipertahankan (Eiben and Smith 2015). Crossover mengkombinasikan materi genetik dari dua orang tua untuk menghasilkan keturunan baru, sedangkan mutasi menerapkan perubahan kecil secara acak pada kromosom dengan probabilitas rendah untuk mempertahankan keberagaman populasi (Goldberg 1989).

$P(\text{mutasi gen}) = p_m$	(2.4)
------------------------------	-------

Keterangan :

$P(\text{mutasi gen})$ = probabilitas suatu gen mengalami mutasi;

p_m = mutation rate atau laju mutasi.

Kelemahan utama GA adalah kerentanan terhadap konvergensi prematur, yaitu kondisi di mana keberagaman populasi berkurang terlalu cepat sehingga algoritma dapat terperangkap pada optima lokal (De Jong 2006). GA juga cenderung kurang efisien dalam fase eksploitasi lokal — meskipun unggul dalam eksplorasi ruang pencarian yang luas, GA kurang efektif dalam memperhalus detail solusi pada skala kecil yang memerlukan pencarian lokal yang presisi.

Karakteristik tersebut menjadikan GA sesuai sebagai komponen eksplorasi global dalam penelitian ini, terutama karena ruang solusi penjadwalan tidur bersifat besar dan diskrit.

2.1.7 Hill Climbing

Hill Climbing (HC) adalah algoritma pencarian lokal yang bergerak secara iteratif dari solusi saat ini ke solusi tetangga (neighbor) yang memiliki nilai fitness lebih tinggi, dan berhenti ketika tidak ditemukan tetangga yang lebih baik (Russell et al. 2021). Berbeda dengan GA yang beroperasi pada populasi, HC beroperasi pada satu solusi tunggal dan merupakan strategi eksploitasi lokal yang efisien dari segi memori dan waktu komputasi per evaluasi.

jika $f(x') > f(x)$, maka $x \leftarrow x'$	(2.5)
--	-------

Keterangan :

$f(x)$ = fitness solusi saat ini;

$f(x')$ = fitness solusi tetangga.

x = solusi saat ini;

x' = solusi tetangga;

Terdapat beberapa varian HC. Steepest ascent hill climbing mengevaluasi seluruh tetangga dan memilih yang terbaik pada setiap langkah. First-choice hill climbing memilih tetangga pertama yang menunjukkan perbaikan untuk efisiensi komputasi. Random restart hill climbing menjalankan HC dari beberapa titik awal berbeda untuk mengurangi dampak optima lokal (Russell et al. 2021). Kelemahan utama HC adalah sifatnya yang greedy dan myopic: algoritma mudah terjebak pada optima lokal, dataran tinggi (plateau), dan punggung

pegunungan (ridge) dalam lanskap fitness (Hoos and Stützle 2005), sehingga lebih cocok digunakan sebagai komplemen algoritma berbasis populasi daripada sebagai pengganti.

Oleh karena itu, Hill Climbing diposisikan sebagai mekanisme penyempurnaan lokal terhadap solusi yang sebelumnya telah dihasilkan oleh Genetic Algorithm.

2.1.8 Genetic Algorithm dan Hill Climbing

Genetic Algorithm dan Hill Climbing (MA) adalah kerangka optimasi hibrida yang menggabungkan eksplorasi global berbasis populasi dari Evolutionary Algorithm (EA) dengan eksploitasi lokal dari prosedur pencarian lokal seperti Hill Climbing atau Simulated Annealing (Moscato 1989). Istilah "memetik" diadopsi dari konsep meme Dawkins sebagai analogi unit informasi budaya yang beradaptasi, sebanding dengan gen dalam evolusi genetis. Sinergi antara GA sebagai penjelajah global dan HC sebagai penghalus lokal merupakan motivasi utama arsitektur ini.

Neri dan Cotta (Neri and Cotta 2012) dalam survei komprehensif mereka melaporkan bahwa pada sebagian besar kelas masalah optimasi kombinatorial yang diuji, MA mampu menghasilkan solusi berkualitas tinggi dengan jumlah evaluasi fungsi fitness yang lebih efisien dibandingkan GA murni atau pencarian lokal yang berdiri sendiri. Performa relatif MA tetap bergantung pada kesesuaian antara prosedur pencarian lokal yang dipilih dengan struktur masalah (Merz and Freisleben 2000).

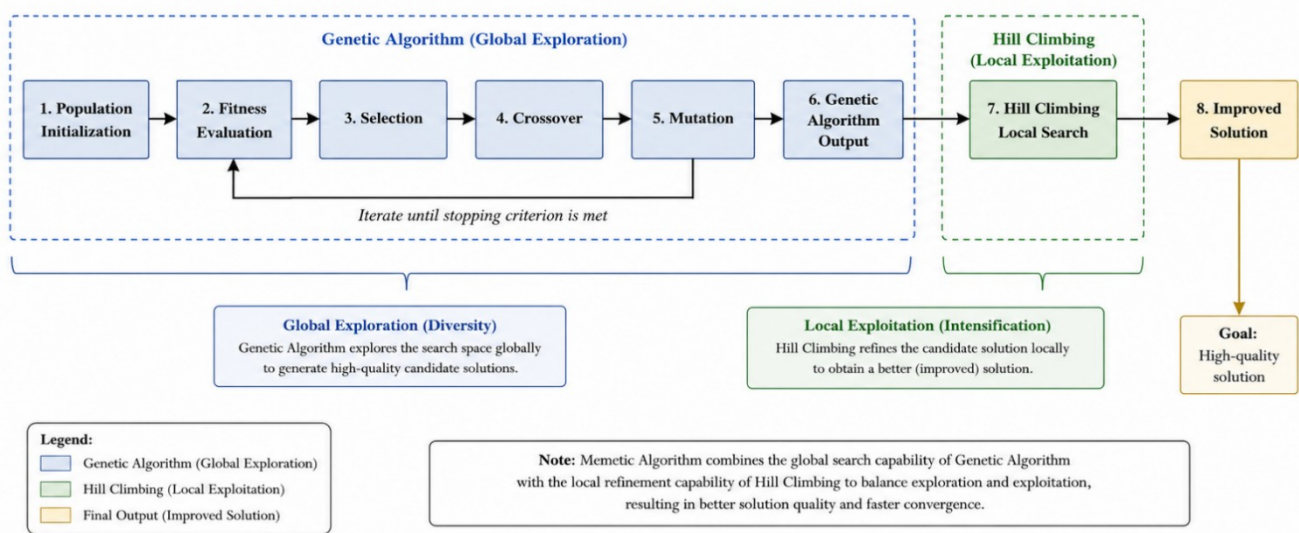
$$x_{final} = HC \left(GA \left(X_0 \right) \right) \quad (2.6)$$

Keterangan :

- X_0 = populasi awal;
- $GA(X_0)$ = solusi terbaik yang dihasilkan Genetic Algorithm;
- HC = proses Hill Climbing;
- x_{final} = solusi akhir hasil optimasi hibrida.

Terdapat dua strategi integrasi utama dalam MA (Neri and Cotta 2012). Strategi Lamarckian mengganti kromosom asli dengan versi yang telah diperbaiki oleh pencarian lokal, sehingga perbaikan lokal langsung diwariskan ke generasi berikutnya dan memengaruhi komposisi genetis populasi. Strategi Baldwinian mengevaluasi fitness berdasarkan solusi yang diperbaiki tanpa mengubah kromosom asli, sehingga pencarian lokal memengaruhi seleksi tetapi tidak mengubah genotipe. Pemilihan antara kedua strategi ini berdampak pada kecepatan konvergensi dan keberagaman populasi.

Kombinasi inilah yang menjadi dasar pemilihan Genetic Algorithm dan Hill Climbing sebagai metode utama dalam penelitian ini.



Gambar 2. 3 Genetic Algorithm dan Hill Climbing

2.1.9 Fungsi Fitness Multikriteria

Fungsi fitness mengkuantifikasi kualitas setiap solusi kandidat sehingga dapat dibandingkan dan diseleksi dalam kerangka GA maupun MA. Dalam masalah optimasi multikriteria, di mana terdapat beberapa tujuan yang tidak selalu konvergen, perancangan fungsi fitness yang tepat menentukan arah evolusi populasi (Deb 2001). Dua pendekatan utama adalah: (1) agregasi linier berbobot, yang menggabungkan beberapa objektif menjadi satu nilai skalar dengan bobot yang mencerminkan kepentingan relatif masing-masing; dan (2) optimasi berbasis dominansi Pareto, yang memelihara sejumlah solusi non-dominan sebagai aproksimasi front Pareto (Deb 2001)

Pada penelitian ini, fitness komposit digunakan untuk membandingkan kandidat berdasarkan pemulihan tidur, durasi tidur, penalti kualitas jadwal, dan regularitas waktu tidur utama. Definisi serta formulasi operasional yang diterapkan dalam program dijelaskan pada subbab Evaluasi Fitness di Bab III.

Nilai fitness dimaksimalkan selama proses optimasi. Komponen dan bobot aktual mengikuti konfigurasi program yang dijelaskan pada Bab III.

2.1.10 Penanganan Kendala (Constraint Handling)

Penanganan kendala merupakan salah satu tantangan utama dalam penerapan GA pada masalah optimasi berkendala (constrained optimization problems, COP) (Coello Coello 2002). Tiga kelompok pendekatan yang lazim digunakan adalah: (1) pendekatan berbasis penalti yang menambahkan penalti pada fitness untuk setiap pelanggaran kendala, sehingga secara implisit mengarahkan pencarian menuju wilayah solusi layak; (2) pendekatan berbasis operator yang merancang crossover dan mutasi sedemikian rupa sehingga keturunan yang dihasilkan selalu layak; dan (3) mekanisme repair constraint yang memperbaiki solusi tidak layak menjadi layak sebelum evaluasi fitness dilakukan (Michalewicz and Nazhiyath, n.d.)

Mekanisme repair umumnya efektif apabila terdapat prosedur deterministik yang efisien untuk mengubah solusi tidak layak menjadi layak, dan sering menghasilkan konvergensi lebih cepat karena seluruh individu yang dievaluasi berada di dalam ruang solusi layak. Pendekatan berbasis penalti, di sisi lain, lebih fleksibel dan mudah

diimplementasikan namun memerlukan kalibrasi koefisien penalti yang cermat (Coello Coello 2002). Detail mekanisme yang diterapkan pada sistem ini diuraikan pada Bab III.

2.1.11 Baseline dalam Evaluasi Algoritma

Dalam evaluasi algoritma optimasi, baseline digunakan sebagai metode pembandingan untuk menilai apakah pendekatan yang diusulkan benar-benar memberikan peningkatan kinerja. Baseline tidak selalu dirancang untuk menghasilkan solusi terbaik, tetapi berfungsi sebagai titik acuan yang sederhana dan mudah dipahami dalam proses perbandingan.

Pada penelitian ini digunakan dua baseline, yaitu greedy schedule dan fixed schedule. Greedy schedule merepresentasikan pendekatan heuristik sederhana yang memilih keputusan terbaik secara lokal berdasarkan slot tidur yang tersedia, tanpa mempertimbangkan dampak antarhari secara menyeluruh. Sementara itu, fixed schedule merepresentasikan pola tidur konstan yang sama setiap hari sebagai pembandingan terhadap jadwal adaptif yang dihasilkan algoritma. Penggunaan kedua baseline tersebut memungkinkan evaluasi apakah Genetic Algorithm dan Hill Climbing mampu menghasilkan jadwal yang lebih baik dari pendekatan sederhana maupun pola tetap.

2.1.12 Representasi Waktu dan Aktivitas

Satu hari direpresentasikan sebagai 48 slot waktu diskret dengan durasi 30 menit per slot. Slot ke-0 merepresentasikan pukul 00.00–00.30, slot ke-1 merepresentasikan pukul 00.30–01.00, dan seterusnya hingga slot ke-47 yang merepresentasikan pukul 23.30–24.00. Dengan representasi ini, seluruh aktivitas harian dapat diproses secara diskret oleh algoritma optimasi tanpa memerlukan pembulatan atau interpolasi waktu.

$$slot = (jam \times 2) + \frac{menit}{30}$$

(2.7)

Keterangan:

slot = indeks slot waktu;

jam = jam dalam format 24 jam;

menit = menit pada waktu tertentu.



Gambar 2. 1 Representasi 48 Slot Harian

Sumber: Diolah penulis

Solusi jadwal harian direpresentasikan sebagai kromosom berbentuk daftar bilangan bulat sepanjang 48 elemen. Setiap elemen, yang disebut gen, menyatakan jenis aktivitas pada slot waktu yang bersangkutan. Kode aktivitas yang digunakan dalam sistem ditunjukkan pada Tabel 2.1.

Tabel 2. 1 Kode Aktivitas pada Representasi Kromosom

Kode	Aktivitas
0	Tidur (tidur utama)
1	Kerja
2	Nap (tidur siang)
3	Santai (waktu luang)
4	Kuliah

$$C = [g_0, g_1, g_2, \dots, g_{47}], g_i \in \{0, 1, 2, 3, 4\}$$

(2.8)

Keterangan:

- C = kromosom jadwal harian;
 g_i = gen pada slot ke- i ;
 i = indeks slot dari 0 sampai 47;
 $\{0, 1, 2, 3, 4\}$ = himpunan kode aktivitas.



Gambar 2. 2 Representasi Kromosom Jadwal Harian

Sumber: Diolah penulis

Representasi kromosom dipilih karena kompatibel secara langsung dengan operator Genetic Algorithm (seleksi, crossover, mutasi) maupun dengan prosedur repair chromosome. Dengan representasi ini, setiap operator dapat diimplementasikan sebagai manipulasi sederhana pada daftar bilangan bulat, sementara kode aktivitas tetap bermakna secara semantis bagi sistem.

2.1.13 Fungsi Fitness dan Evaluasi Solusi

Fungsi fitness adalah ukuran komputasional untuk membandingkan kualitas kandidat jadwal. Dalam penelitian ini, fitness menggabungkan manfaat pemulihan, durasi tidur, penalti terhadap pola yang tidak diinginkan, dan konsistensi waktu mulai tidur utama antarhari. Rincian komponen, bobot, dan persamaan yang benar-benar diterapkan dijelaskan pada Bab III.

2.1.14 Penanganan Kendala

Pada masalah penjadwalan, kendala keras menentukan aktivitas yang harus tetap pada slotnya, sedangkan kendala lunak menyatakan preferensi kualitas yang dapat dinilai melalui fungsi objektif. Mekanisme repair digunakan untuk mengembalikan kandidat ke bentuk yang memenuhi aturan operasional setelah inisialisasi dan operator evolusioner. Definisi himpunan kendala dan urutan repair SORA dijelaskan pada Bab III.

2.1.15 Parameter Algoritma

Ukuran populasi, laju crossover, laju mutasi, jumlah generasi, dan batas iterasi pencarian lokal mengendalikan cakupan serta biaya komputasi pencarian. Nilai parameter merupakan bagian dari rancangan eksperimen, sehingga konfigurasi final dan status kalibrasinya dilaporkan pada Bab III dan Bab IV.

2.2 Tinjauan Sumber Pustaka

Subbab ini menyajikan tinjauan terhadap penelitian-penelitian terdahulu yang relevan dengan topik penelitian ini, meliputi studi di bidang ilmu tidur komputasional, algoritma metaheuristik untuk penjadwalan, dan aplikasi optimasi pada domain kesehatan. Tinjauan disusun untuk mengidentifikasi landasan empiris, metode yang telah diuji, dan kesenjangan yang masih ada. Tabel 2.1 merangkum penelitian-penelitian tersebut beserta metode, kontribusi, dan keterbatasannya.

Tabel 2. 1 State of The Art

No	Penelitian	Metode	Hasil Penelitian	Keterbatasan
1	(Borbély 1982)	Pemodelan matematis two-process model; simulasi respons EEG gelombang lambat	Membangun kerangka teoritis regulasi tidur homeostatis-sirkadian yang menjadi fondasi pemodelan komputasional tidur; menjelaskan mekanisme akumulasi dan peluruhan tekanan tidur	Bersifat teoritis-fisiologis; tidak ada komponen optimasi penjadwalan otomatis
2	(Van Dongen et al. 2003)	Studi eksperimental dosis-respons; Psychomotor Vigilance Task (PVT); desain within-subject terkontrol	Mengkuantifikasi dampak kumulatif restriksi tidur kronik pada kinerja kognitif; membuktikan adaptasi perseptual yang mengaburkan defisit subjektif; menjadi acuan standar pemodelan sleep debt	Dilakukan di lingkungan laboratorium terkontrol; tidak mempertimbangkan variabilitas jadwal multi-hari atau faktor regularitas antarahari
3	(Wittmann et al. 2006)	Survei kronobiologi berskala besar; Munich Chronotype Questionnaire (MCTQ); analisis korelasional	Memperkenalkan dan mengoperasionisasikan konsep social jetlag; menghubungkan ketidakselarasan sirkadian-sosial dengan konsumsi stimulan dan penurunan kualitas tidur subjektif	Bersifat observasional cross-sectional; tidak ada sistem penjadwalan atau intervensi otomatis; kausalitas belum ditetapkan
4	(Phillips et al. 2017)	Pemodelan matematis SRI; analisis kohort akselerometri mahasiswa; korelasi dengan performa akademik	Memperkenalkan SRI sebagai metrik kuantitatif regularitas tidur; menunjukkan korelasi antara konsistensi pola tidur-bangun dengan performa akademik dan keterlambatan fase sirkadian	Bersifat deskriptif-analitik; SRI tidak diintegrasikan ke dalam sistem optimasi atau penjadwalan otomatis manapun

No	Penelitian	Metode	Hasil Penelitian	Keterbatasan
5	(Roenneberg et al. 2012)	Survei kronobiologi multi-negara; analisis korelasi social jetlag dengan indeks massa tubuh	Mengkuantifikasi hubungan antara social jetlag dan kelebihan berat badan; memperluas basis epidemiologis konsep social jetlag ke domain metabolik	Tidak ada komponen penjadwalan otomatis; tidak mempertimbangkan strategi mitigasi berbasis optimasi
6	(Aickelin and Dowsland 2004)	Genetic Algorithm dengan representasi integer; penanganan kendala berbasis penalti; nurse scheduling problem	Membuktikan efektivitas GA pada masalah penjadwalan berkendalikan kompleks di domain kesehatan; GA mampu melampaui heuristik manual dalam kualitas solusi pada kasus rumah sakit nyata	Domain perawat, bukan tidur personal; tidak mempertimbangkan faktor biologis seperti sleep debt atau ritme sirkadian
7	(Huang, Mariani, and Redline 2020)	Analisis kohort MESA (Multi-Ethnic Study of Atherosclerosis); aktigrafi multi-malam; pemodelan risiko prospektif	Menghubungkan iregularitas tidur-bangun secara prospektif dengan peningkatan risiko kejadian kardiovaskular mayor, independen dari durasi tidur, dalam kohort multi-etnik skala besar	Bersifat epidemiologis; tidak ada sistem optimasi; iregularitas diukur secara observasional, bukan diintervensi
8	(Neri and Cotta 2012)	Survei sistematik Genetic Algorithm dan Hill Climbing; benchmarking komparatif GA vs MA pada berbagai kelas masalah NP-hard	Menunjukkan bahwa pada sebagian besar kelas masalah yang diuji, MA menghasilkan solusi lebih kompetitif dibandingkan GA murni dengan evaluasi yang lebih efisien; mengklasifikasikan strategi integrasi Lamarckian dan Baldwinian	Survei umum; tidak diaplikasikan secara spesifik pada domain kesehatan atau penjadwalan tidur
9	(Windred et al. 2024)	Analisis kohort longitudinal UK Biobank; akselerometri; pemodelan risiko mortalitas dengan pengendalian perancu	Melaporkan bahwa SRI rendah berhubungan dengan risiko mortalitas lebih kuat dibandingkan durasi tidur yang tidak adekuat; mendorong pergeseran perhatian klinis dari durasi ke regularitas tidur	Bersifat observasional; tidak ada intervensi atau sistem optimasi; mekanisme kausal belum sepenuhnya ditetapkan
10	(Boivin,	Tinjauan	Mengkompilasi bukti	Bersifat review;

No	Penelitian	Metode	Hasil Penelitian	Keterbatasan
	Boudreau, and Kosmadopoulos (2022)	sistematis dan meta-analisis; shift work dan gangguan ritme sirkadian; analisis risiko kardiovaskular dan metabolik	epidemiologis dampak pola kerja tidak teratur terhadap kesehatan; mendukung perlunya intervensi berbasis regularitas jadwal pada populasi pekerja dengan pola tidak teratur	tidak mengajukan sistem penjadwalan otomatis; rekomendasi bersifat naratif dan umum
11	(Choi et al. 2018)	Review dan evaluasi aplikasi smartphone untuk sleep self-management	Menunjukkan bahwa aplikasi tidur digital telah banyak digunakan untuk mendukung pemantauan dan pengelolaan tidur mandiri	Tidak membahas optimasi jadwal tidur berbasis metaheuristik, tidak menerapkan penalti regularitas yang terinspirasi konsep SRI sebagai tujuan optimasi, dan tidak memodelkan kendala aktivitas harian secara kombinatorial

Sumber: Diolah dari berbagai pustaka, 2026

2.3 Gap Penelitian dan Posisi Penelitian

Berdasarkan landasan teori pada Subbab 2.1 dan tinjauan sumber pustaka pada Subbab 2.2, terdapat tiga kesenjangan penelitian (research gap) yang dapat diidentifikasi.

Pertama, berbagai penelitian telah menunjukkan pentingnya regularitas tidur terhadap kualitas fungsi harian dan kondisi kesehatan. Namun, penerapan aspek regularitas tidur sebagai pertimbangan operasional dalam sistem optimasi jadwal tidur otomatis masih relatif terbatas. Penelitian ini menggunakan konsep regularitas tidur tersebut sebagai dasar perancangan penalti terhadap perubahan waktu mulai tidur utama antarhari, tanpa mengklaim melakukan perhitungan nilai SRI klinis secara penuh.

Kedua, penelitian di bidang optimasi penjadwalan berkendalikan, seperti nurse scheduling dengan GA yang dilakukan oleh Aickelin dan Dowsland (Aickelin and Dowsland 2004), telah menunjukkan potensi pendekatan metaheuristik pada masalah-masalah sejenis di domain kesehatan. Namun penerapan pendekatan ini pada domain penjadwalan tidur personal dengan kendala biologis yang relevan secara klinis — mencakup sleep debt, fatigue index, dan regularitas sirkadian — masih relatif terbatas dalam literatur yang tersedia.

Ketiga, berbagai aplikasi digital untuk manajemen tidur umumnya berfokus pada pemantauan, edukasi, atau dukungan pengelolaan tidur secara mandiri, tetapi belum secara khusus diarahkan sebagai sistem optimasi jadwal tidur personal berbasis metaheuristik yang mempertimbangkan kendala dan preferensi individual secara kombinatorial (Choi et al.

2018). Sistem-sistem ini belum memanfaatkan pendekatan metaheuristik yang mampu menjelajahi ruang solusi luas tanpa memerlukan asumsi konveksitas atau diferensiabilitas fungsi tujuan.

Penelitian ini mengusulkan optimasi jadwal tidur mingguan dengan penalti regularitas berbasis pergeseran waktu tidur utama antarhari. Pendekatan tersebut terinspirasi oleh pentingnya regularitas tidur yang dijelaskan dalam literatur SRI, namun tidak dimaksudkan sebagai perhitungan nilai SRI klinis penuh.

BAB III

METODOLOGI PENELITIAN

3.1 Jenis dan Rancangan Penelitian

Penelitian ini merupakan penelitian implementatif dengan pendekatan eksperimen komputasional. Disebut implementatif karena tujuan utama penelitian adalah merancang, membangun, dan menguji sistem optimasi jadwal tidur berbasis algoritma pada program yang dapat dijalankan; bukan menghasilkan teori baru maupun melakukan survei terhadap fenomena empiris. Pendekatan eksperimen komputasional digunakan karena evaluasi dilakukan melalui serangkaian percobaan terprogram terhadap dataset aktivitas subjek, mencakup kalibrasi parameter, optimasi jadwal, dan perbandingan hasil terhadap metode baseline.

Metode utama yang diterapkan adalah Genetic Algorithm dan Hill Climbing, yaitu algoritma hibrida yang menggabungkan Genetic Algorithm (GA) sebagai mekanisme eksplorasi global dengan Hill Climbing (HC) sebagai mekanisme eksploitasi lokal. GA bertugas menjelajahi ruang solusi yang luas melalui mekanisme evolusi berbasis populasi, sedangkan HC memperhalus solusi terbaik hasil evolusi melalui pencarian lokal yang presisi. Dalam implementasi sistem ini, kedua metode digabungkan secara sekuensial. Genetic Algorithm terlebih dahulu digunakan untuk menjelajahi ruang solusi dan menghasilkan kandidat jadwal terbaik, kemudian Hill Climbing diterapkan pada solusi terbaik tersebut untuk melakukan penyempurnaan lokal.

Objek penelitian berupa jadwal tidur dan aktivitas harian subjek dalam horizon tujuh hari. Jadwal direpresentasikan sebagai kromosom diskret sepanjang 48 gen, di mana setiap gen merepresentasikan satu slot waktu berdurasi 30 menit. Aktivitas yang dimodelkan meliputi tidur utama, kerja, nap, santai, dan kuliah. Sistem menghasilkan rekomendasi jadwal yang mempertimbangkan aktivitas wajib, target durasi tidur, perubahan fatigue akibat aktivitas, manfaat pemulihan tidur, serta regularitas waktu mulai tidur antarmhari.

Secara keseluruhan, tahapan penelitian meliputi penyediaan dataset, pra-pemrosesan dan representasi slot wajib, perancangan fitness, implementasi repair chromosome, Genetic Algorithm dan Hill Climbing, optimasi mingguan dengan carry-over, perbandingan dengan baseline, serta evaluasi multi-subjek. Aplikasi menyediakan mekanisme kalibrasi, tetapi kalibrasi dilewati pada eksekusi final.

3.2 Data Penelitian

Data penelitian diperoleh melalui kuesioner terhadap sejumlah responden yang memenuhi kriteria penelitian. Kuesioner mengumpulkan informasi aktivitas wajib harian serta kebutuhan istirahat subjek. Jawaban responden kemudian dikonversi menjadi indeks slot waktu dalam format CSV, dengan satu slot merepresentasikan interval 30 menit.

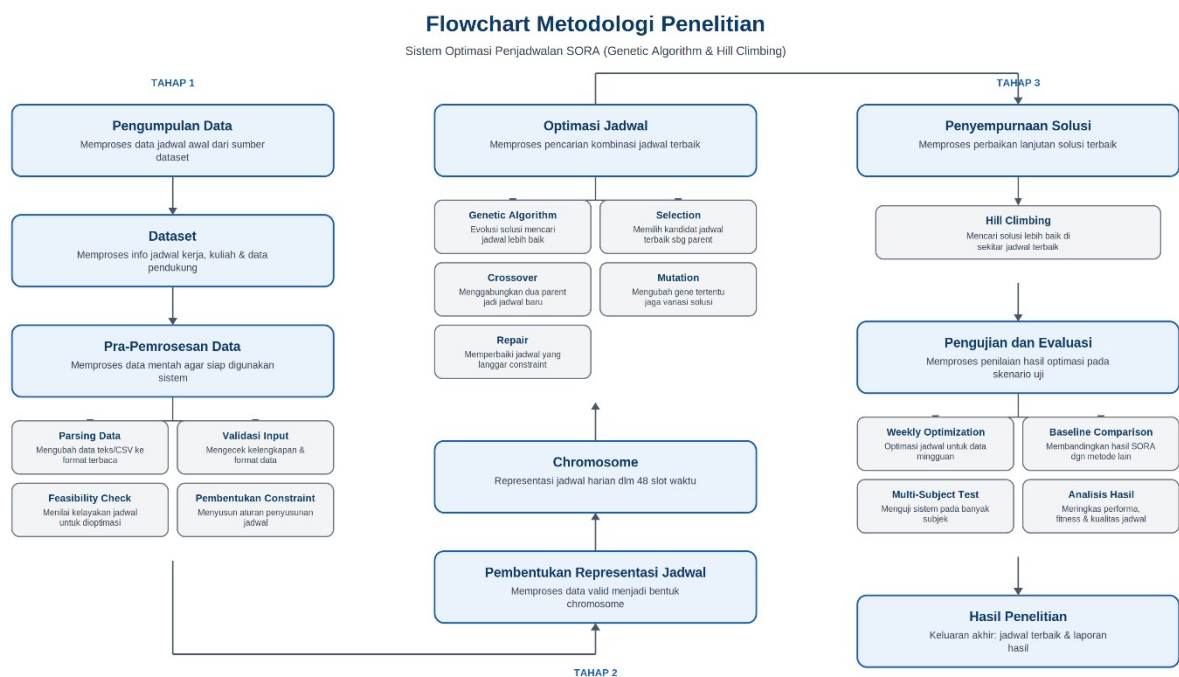
Data yang digunakan dalam penelitian ini berupa dataset aktivitas subjek dalam format CSV. Setiap baris pada dataset merepresentasikan satu subjek, dan setiap pasang kolom merepresentasikan aktivitas subjek pada hari tertentu dalam seminggu. Kolom-kolom utama yang digunakan adalah sebagai berikut.

Nama, yaitu identitas subjek yang digunakan untuk penelusuran data.

Kerja_Senin hingga Kerja_Minggu, yaitu daftar indeks slot waktu aktivitas kerja pada masing-masing hari.

Kuliah_Senin hingga Kuliah_Minggu, yaitu daftar indeks slot waktu aktivitas kuliah pada masing-masing hari.

Dataset pengujian final adalah ekspor mentah Google Forms `kuisoner (Jawaban) - Form Responses 1.csv`, yang memuat 63 baris data, 63 responden unik, dan 27 kolom sumber. Eksperimen utama memakai seluruh 63 responden selama tujuh hari. Kalibrasi dilewati pada eksekusi final dengan opsi `--skip-calibration`; karena itu angka hasil Bab IV tidak ditafsirkan sebagai hasil grid search pada dataset pengujian.



Gambar 3. 3 Alur Metodologi Penelitian

Sumber: Diolah penulis

3.3 Alur Penelitian

Penelitian ini dilaksanakan melalui serangkaian tahapan yang tersusun secara sistematis, dimulai dari pengumpulan data hingga evaluasi hasil optimasi. Setiap tahapan dirancang agar proses penelitian dapat berjalan secara terstruktur serta menghasilkan sistem optimasi jadwal tidur dan aktivitas yang sesuai dengan tujuan penelitian. Alur penelitian yang digunakan ditunjukkan pada Gambar 3.1.

Tahap pertama adalah pengumpulan data penelitian. Pada tahap ini dilakukan pengumpulan data aktivitas harian pengguna melalui formulir isian yang kemudian dikonversi ke dalam berkas berformat CSV. Data tersebut memuat informasi mengenai

jadwal aktivitas wajib, seperti jam kerja dan jam kuliah, serta preferensi pengguna yang meliputi target durasi tidur, waktu aktivitas, dan batasan-batasan lain yang diperlukan selama proses optimasi. Data yang telah dikumpulkan menjadi masukan utama bagi sistem.

Tahap kedua adalah pra-pemrosesan data (pre-processing). Sistem melakukan pembacaan terhadap data CSV untuk memastikan seluruh informasi memiliki format yang sesuai. Selanjutnya seluruh aktivitas dikonversi ke dalam representasi waktu diskret sebanyak 48 slot per hari dengan interval 30 menit setiap slot. Pada tahap ini juga dilakukan identifikasi terhadap aktivitas yang bersifat wajib (mandatory), pembentukan slot transisi antaraktivitas, serta pemeriksaan kelayakan data (feasibility check). Apabila ditemukan konflik jadwal atau data yang tidak memenuhi ketentuan sistem, proses optimasi tidak dilanjutkan hingga data diperbaiki.

Setelah data dinyatakan valid, sistem membangkitkan populasi kandidat yang mempertahankan aktivitas wajib. Setiap kandidat dievaluasi dengan fungsi fitness, kemudian diproses menggunakan seleksi, crossover, mutasi, repair, dan Hill Climbing. Rincian matematis dan urutan operasional masing-masing proses disajikan pada subbab yang membahas proses tersebut.

Tahap berikutnya adalah optimasi jadwal tidur mingguan. Optimasi dilakukan secara berurutan selama tujuh hari. Hasil optimasi pada suatu hari tidak berdiri sendiri, tetapi digunakan sebagai kondisi awal pada hari berikutnya. Informasi seperti tingkat kelelahan akhir dan waktu mulai tidur utama diteruskan (carry-over) sehingga sistem mampu menghasilkan jadwal yang lebih konsisten sepanjang minggu.

Pipeline menyediakan fasilitas kalibrasi parameter melalui grid search. Namun, pada eksperimen final fasilitas tersebut dilewati dengan opsi `--skip-calibration`; parameter eksperimen final karena itu mengikuti konfigurasi `default_fallback` yang dicatat pada `METADATA.json`.

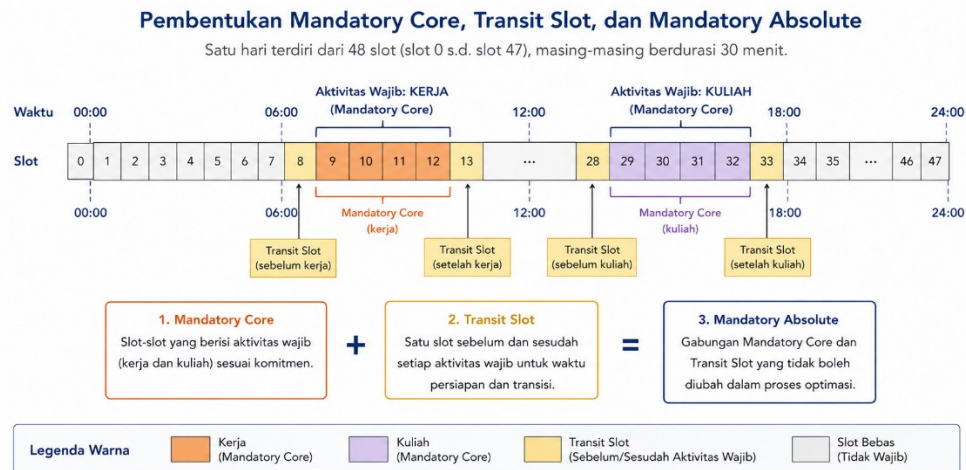
Konfigurasi final yang sama digunakan pada tahap Baseline Comparison. Genetic Algorithm dan Hill Climbing dibandingkan dengan Greedy Schedule dan Fixed Schedule menggunakan masukan aktivitas yang sama.

Selanjutnya dilakukan evaluasi sistem melalui tiga skenario pengujian, yaitu pengujian optimasi mingguan (Weekly Optimization), pengujian perbandingan dengan metode baseline (Baseline Comparison), dan pengujian terhadap beberapa subjek dengan karakteristik aktivitas yang berbeda (Multi-Subject Test). Hasil setiap pengujian dianalisis berdasarkan nilai fitness, durasi tidur yang diperoleh, tingkat kelelahan akhir, serta konsistensi pola tidur yang dihasilkan.

Tahap terakhir adalah implementasi dan penyajian hasil. Seluruh algoritma diimplementasikan menggunakan bahasa pemrograman Python dengan antarmuka berbasis web (React dan Streamlit). Sistem menghasilkan jadwal tidur yang telah dioptimalkan beserta laporan evaluasi yang memuat nilai fitness, statistik tidur, serta visualisasi jadwal harian dan mingguan. Seluruh hasil tersebut kemudian dianalisis untuk menjawab tujuan penelitian dan menarik kesimpulan mengenai kinerja Genetic Algorithm dan Hill Climbing dalam mengoptimalkan jadwal tidur pada pekerja dengan pola aktivitas yang tidak teratur.

3.4 Pra-Pemrosesan Data

Pra-pemrosesan data dilakukan sebelum proses optimasi agar data aktivitas dari file CSV dapat dikonversi menjadi struktur data yang siap digunakan oleh algoritma. Tahapan ini meliputi parsing slot aktivitas, pembentukan mandatory core, pembentukan transit slot, pembentukan mandatory absolute, dan feasibility check.



Gambar 3. 4 Mandatory Core, Transit Slot, dan Mandatory Absolute

Sumber: Diolah penulis

3.4.1 Parsing Slot Aktivitas

Parsing slot dilakukan untuk mengubah nilai pada kolom Kerja dan Kuliah di file CSV menjadi dua daftar bilangan bulat, yaitu kerja_slots dan kuliah_slots. Setiap daftar hanya dinyatakan valid apabila seluruh elemennya berupa bilangan bulat pada rentang 0 hingga 47. Jika isi kolom tidak dapat diparse atau mengandung nilai di luar rentang tersebut, parser mengembalikan daftar kosong agar data tidak diproses sebagai slot aktivitas yang salah.

3.4.2 Pembentukan Mandatory Core

Mandatory core merupakan gabungan dari kerja_slots dan kuliah_slots. Slot yang termasuk mandatory core diperlakukan sebagai aktivitas wajib yang tidak boleh diubah oleh proses optimasi. Dalam inisialisasi kromosom, slot kerja ditetapkan dengan kode aktivitas 1 dan slot kuliah ditetapkan dengan kode aktivitas 4.

$$\text{MandatoryCore} = \text{KerjaSlots} \cup \text{KuliahSlots}$$

(3.1)

Keterangan:

MandatoryCore = himpunan slot wajib utama;
KerjaSlots = slot aktivitas kerja;
KuliahSlots = slot aktivitas kuliah.

3.4.3 Pembentukan Transit Slot

Transit slot adalah slot penyangga yang ditambahkan satu posisi sebelum dan satu posisi sesudah setiap elemen mandatory core, sepanjang slot tersebut belum menjadi bagian dari mandatory core. Transit slot diberi kode aktivitas santai (3). Penambahan ini bertujuan agar tidur tidak berbatasan langsung dengan aktivitas wajib, sehingga transisi antaraktivitas menjadi lebih realistis secara biologis dan menghindari kromosom yang menempatkan tidur tepat berdampingan dengan jam kerja atau kuliah.

$$TransitSlots = \{j / j \in \{i-1, i+1\}, i \in MandatoryCore, 0 \leq j \leq 47\} \quad (3.2)$$

Keterangan:

TransitSlots = himpunan slot transisi;
i = indeks slot pada mandatory core;
j = slot transisi hasil perluasan dari slot wajib.

3.4.4 Pembentukan Mandatory Absolute

Mandatory absolute merupakan gabungan antara mandatory core dan transit slot. Seluruh slot yang termasuk dalam mandatory absolute tidak boleh dimodifikasi oleh operator evolusi (crossover dan mutasi) maupun oleh prosedur repair chromosome. Dengan pembatasan ini, sistem menjamin bahwa aktivitas wajib beserta slot transisinya selalu dipertahankan sepanjang proses optimasi.

$$MandatoryAbsolute = MandatoryCore \cup TransitSlots \quad (3.3)$$

Keterangan:

MandatoryAbsolute = himpunan slot wajib absolut;
MandatoryCore = slot kerja dan kuliah;
TransitSlots = slot transisi.

3.4.5 Feasibility Check

Feasibility check digunakan untuk memastikan bahwa jadwal harian masih memiliki ruang yang memadai untuk proses optimasi tidur. Dalam implementasi sistem, pemeriksaan

kelayakan dilakukan berdasarkan jumlah slot yang termasuk dalam *mandatory_absolute*. Apabila jumlah slot wajib melebihi 36 dari total 48 slot harian, jadwal dinyatakan tidak layak karena tidak lagi menyediakan ruang minimum yang memadai untuk tidur.

$$|MandatoryAbsolute| \leq 36 \quad (3.4)$$

Keterangan:

$|MandatoryAbsolute|$ = jumlah slot wajib absolut;

36 = batas maksimum slot wajib agar jadwal masih layak dioptimasi.

Selain feasibility check, sistem juga memiliki mekanisme *survival_mode* yang diaktifkan apabila blok slot bebas terpanjang dalam satu hari kurang dari 12 slot dan terdapat aktivitas wajib pada hari tersebut. Berbeda dari feasibility check, *survival_mode* tidak menghentikan optimasi, melainkan menandai bahwa jadwal berada dalam kondisi sangat padat sehingga beberapa aturan kualitas tidur perlu dilonggarkan agar solusi tetap dapat dicari.

$$SisaSlotBebas = 48 - |MandatoryAbsolute| \quad (3.5)$$

Keterangan:

SisaSlotBebas = jumlah slot bebas yang masih digunakan;

48 = total slot dalam satu hari;

$|MandatoryAbsolute|$ = jumlah slot yang tidak boleh dimodifikasi.

Slot kerja dan kuliah menjadi aktivitas wajib. Sistem menambahkan slot transisi tepat sebelum atau sesudah aktivitas wajib apabila slot tersebut masih berada dalam hari dan bukan aktivitas wajib. Kelayakan dan kondisi jadwal padat diperiksa dari jumlah slot wajib serta rentetan slot bebas terpanjang.

3.5 Prosedur Eksekusi Genetic Algorithm dan Hill Climbing

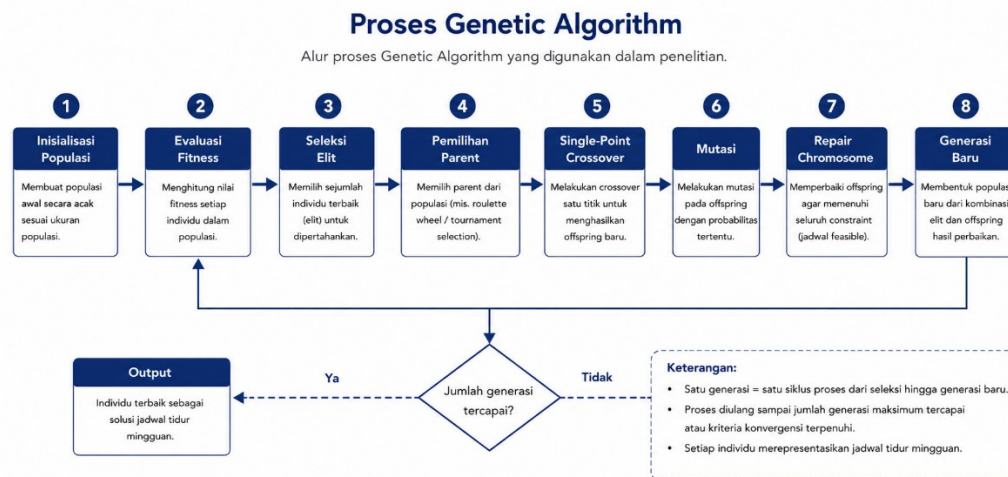
Prosedur eksekusi Genetic Algorithm dan Hill Climbing merupakan tahapan utama dalam penelitian ini yang bertujuan menghasilkan jadwal tidur dan aktivitas dengan kualitas terbaik berdasarkan data yang telah melalui proses pra-pemrosesan. Algoritma yang digunakan menggabungkan Genetic Algorithm sebagai mekanisme pencarian global (global search) dengan Hill Climbing sebagai mekanisme penyempurnaan lokal (local search). Kombinasi kedua metode tersebut memungkinkan sistem mengeksplorasi ruang solusi yang luas sekaligus memperbaiki solusi terbaik yang telah diperoleh sehingga menghasilkan jadwal yang lebih optimal.

Secara umum, proses optimasi diawali dengan pembentukan populasi awal yang terdiri atas sejumlah kandidat jadwal. Setiap kandidat direpresentasikan sebagai kromosom

yang memuat susunan aktivitas selama satu hari dalam bentuk 48 slot waktu. Seluruh kromosom yang dibangkitkan terlebih dahulu melewati proses repair chromosome untuk memastikan bahwa seluruh aktivitas wajib tetap terpenuhi dan tidak terdapat pelanggaran terhadap kendala keras (hard constraints). Setelah dinyatakan layak, setiap kromosom dievaluasi menggunakan fungsi fitness untuk mengetahui tingkat kualitas solusi yang dihasilkan.

Selanjutnya Genetic Algorithm dijalankan secara iteratif selama sejumlah generasi. Pada setiap generasi dilakukan proses seleksi individu terbaik, pembentukan keturunan melalui crossover, mutasi terhadap sebagian gen, serta evaluasi ulang terhadap individu hasil evolusi. Setelah seluruh proses evolusi selesai, solusi terbaik yang diperoleh diteruskan ke tahap Hill Climbing. Tahap ini bertujuan memperbaiki detail jadwal melalui pencarian lokal sehingga diperoleh solusi akhir yang memiliki nilai fitness lebih tinggi dibandingkan hasil Genetic Algorithm saja.

Alur lengkap pelaksanaan Genetic Algorithm dan Hill Climbing pada penelitian ini ditunjukkan pada Gambar 3.6.



Gambar 3. 5 Alur Genetic Algorithm

Sumber: Diolah penulis

3.5.1 Inisialisasi Populasi

Tahap pertama adalah pembentukan populasi awal. Sistem membangkitkan sejumlah kromosom secara acak dengan tetap memperhatikan batasan aktivitas yang telah diperoleh pada tahap pra-pemrosesan data. Aktivitas yang bersifat wajib, seperti jam kerja, kuliah, maupun slot transisi, dipertahankan pada posisi yang telah ditentukan, sedangkan slot yang masih kosong diisi dengan kandidat aktivitas tidur utama, tidur siang (nap), atau waktu luang.

Setelah populasi awal terbentuk, setiap kromosom diperiksa menggunakan mekanisme repair chromosome. Tahap ini memastikan bahwa tidak terdapat konflik aktivitas maupun pelanggaran terhadap seluruh kendala wajib sebelum kromosom dievaluasi lebih lanjut.

Satu individu merepresentasikan jadwal satu hari dalam 48 slot. Setiap slot berdurasi 30 menit dan memakai kode aktivitas yang telah didefinisikan pada subbab representasi solusi.

$x = (x_0, x_1, \dots, x_{47}), x_s \in \{0, 1, 2, 3, 4\}$
$\Delta t = 0,5 \text{ jam per slot}$

3.5.2 Evaluasi Fitness

Seluruh kromosom yang telah dinyatakan layak kemudian dievaluasi menggunakan fungsi fitness. Proses evaluasi dilakukan untuk menentukan kualitas setiap kandidat jadwal berdasarkan beberapa indikator yang telah ditetapkan pada penelitian ini.

Evaluasi mempertimbangkan tingkat pemulihan tidur, durasi tidur aktual, tingkat kelelahan setelah menjalankan seluruh aktivitas, serta konsistensi waktu mulai tidur dibandingkan hari sebelumnya. Hasil evaluasi berupa satu nilai fitness yang digunakan sebagai dasar dalam menentukan individu terbaik selama proses evolusi.

Nilai fitness yang lebih tinggi menunjukkan bahwa kromosom tersebut memiliki kualitas jadwal yang lebih baik sehingga memiliki peluang lebih besar untuk dipertahankan pada generasi berikutnya.

Durasi tidur utama dihitung dari banyaknya gen bernilai tidur. Target tidur operasional mempertimbangkan jumlah tidur observasi bila tersedia; jika tidak tersedia, sistem menggunakan target dasar yang dapat ditambah cicilan sleep debt. Target dibatasi oleh slot bebas yang masih tersedia.

$A = \sum_s I(x_s = 0)$
$H(x) = 0,5A \text{ jam}$
$F = 48 - M $
$T_{\text{sleep}} = \min(O , \max(0, F-2)) \text{ jika } O > 0$
$T_{\text{sleep}} = \min(14+d, \max(0, F-2)) \text{ jika } O = 0$

Fatigue dihitung berurutan dari nilai awal. Kerja, kuliah, dan istirahat menambah fatigue dengan besaran berbeda. Tidur utama mengurangi fatigue secara proporsional terhadap fatigue saat itu dan bobot sirkadian. Nap hanya memberi pemulihan jika berada di jendela yang ditetapkan dan cukup jauh dari akhir tidur utama.

$c(s) = 1,20 \text{ jika } s \leq 12 \text{ atau } s \geq 44; 0,85 \text{ jika } 24 \leq s \leq 32; 0,65 \text{ untuk slot lainnya}$
$f_{s+1} = \max(0, f_s + 0,08I_{\text{work}} + 0,05I_{\text{class}} + 0,03I_{\text{rest}})$
$r_{\text{sleep}}(s) = 0,15 c(s) f_s$
$f_{s+1} = \max(0, f_s(1 - 0,15c(s))) \text{ jika } x_s = 0$
$r_{\text{nap}}(s) = 0,25c(s); f_{s+1} = \max(0, f_s - 0,35) \text{ jika nap valid}$
$f_{s+1} = \max(0, f_s + 0,03) \text{ jika } x_s = 2 \text{ dan } s \notin [24, 33]$
$I_{\text{nap_valid}}(s) = 1 \Leftrightarrow 24 \leq s < 34 \wedge (\text{core_end} = -1 \vee s - \text{core_end} \geq 8)$

Pada kondisi normal, penalti durasi diterapkan terhadap batas minimum, batas maksimum, dan target operasional. Jika jadwal memiliki tidur observasi, perbedaan durasi dan tumpang tindih slot tidur observasi turut dinilai. Pada survival mode, penalti durasi dilonggarkan; jika tidak ada nap, sistem memberikan penalti khusus.

$P_{\text{dur}} = -200I_{\text{normal}}[\max(0, 12-A)^2 + \max(0, A-18)^2 + \max(0, T_{\text{sleep}}-A)^2]$

$P_{\text{obs}} = I(O > 0)[-20(A - O)^2 + 0,35 S \cap O]$
$P_{\text{no- nap}} = -450I_{\text{survival}} I(N=0)$

Penalti dinamis turut membatasi fragmentasi tidur utama, jumlah slot nap, dan jumlah sesi nap. Penalti regularitas membandingkan waktu mulai blok tidur utama terpanjang dengan anchor dari hari sebelumnya menggunakan jarak melingkar. Penalti regularitas diterapkan hanya ketika pergeseran melebihi toleransi dan anchor tidak terhalang aktivitas wajib.

$P_{\text{block}} = -300[\max(0, B - B_{\text{max}})]^2$; $B_{\text{max}} = 3 + I_{\text{survival}}$
$P_{\text{nap-slot}} = -150[\max(0, N - 3)]^2$
$P_{\text{nap-session}} = -150[\max(0, Q - 1)]^2$
$\delta = \min(c - a , 48 - c - a)$
$P_{\text{reg}} = -\min(400, 100\max(0, \delta - 4))I(\delta > 4 \wedge \neg \text{blocked})$

Nilai fitness menggabungkan reward pemulihan dan durasi tidur dengan penalti dinamis serta penalti regularitas. Skor dapat negatif ketika penalti lebih besar daripada reward. Penalti regularitas ini merupakan heuristik waktu mulai tidur utama, bukan perhitungan SRI standar.

$R = \sum_s r_{\text{sleep}}(s) + \sum_s r_{\text{nap}}(s)$
$P_{\text{nap-early}} = -150N_{\text{early}}$
$P_{\text{dyn}} = P_{\text{dur}} + P_{\text{obs}} + P_{\text{block}} + P_{\text{nap-slot}} + P_{\text{nap-session}} + P_{\text{nap-early}} + P_{\text{no- nap}}$
$\text{Fitness}(x) = 0,65[R + 0,2A] + P_{\text{dyn}} + P_{\text{reg}}$

3.5.3 Elitisme dan Seleksi Parent

Setelah seluruh individu dievaluasi, sistem melakukan proses seleksi untuk menentukan kromosom yang akan digunakan sebagai induk (parent) pada proses reproduksi.

Selain seleksi, diterapkan pula mekanisme elitisme untuk mempertahankan sejumlah individu dengan nilai fitness tertinggi agar tetap berada pada populasi generasi berikutnya. Penerapan elitisme bertujuan menjaga agar solusi terbaik yang telah diperoleh tidak hilang akibat proses crossover maupun mutasi.

Dua parent dipilih tanpa pengembalian secara acak dari paling banyak 50 individu berperingkat teratas. Dengan konfigurasi eksperimen utama berukuran 50, parent berasal dari populasi yang telah diurutkan berdasarkan fitness.

Populasi diurutkan dari fitness tertinggi. Sejumlah individu terbaik disalin ke generasi berikutnya tanpa perubahan; parent dipilih tanpa pengembalian dari kelompok berperingkat teratas.

$E(N) = 20$ jika $N > 20$; $E(N) = \max(1, \lfloor N/5 \rfloor)$ jika $N \leq 20$
$N_{\text{parent}} = \min(50, N)$

3.5.4 Crossover

Crossover menggunakan metode single-point crossover. Titik potong dipilih secara acak pada rentang slot 10 hingga 37 untuk menghindari keturunan yang hampir identik dengan salah satu parent. Gen dari parent pertama diambil dari slot 0 hingga titik potong, dan gen dari parent kedua diambil dari titik potong hingga slot 47, sehingga menghasilkan

satu keturunan. Setiap keturunan hasil crossover kemudian diproses oleh repair chromosome sebelum dimasukkan ke populasi berikutnya.

$$Child = Parent_1[0:c] + Parent_2[c:48] \quad (3.6)$$

Keterangan :

Child : kromosom anak;
Parent₁ : kromosom induk pertama;
Parent₂ : kromosom induk kedua;
c : titik potong crossover;
 $[0:c]$: bagian kromosom dari indeks 0 sampai sebelum titik potong;
 $[c:48]$: bagian kromosom dari titik potong sampai indeks 47.

Crossover satu titik menyambungkan bagian awal dari parent pertama dan bagian akhir dari parent kedua. Jika operator crossover tidak aktif, parent pertama disalin sebagai anak. Anak selanjutnya menjalani mutasi dan repair.

$k \in \{10, 11, \dots, 37\}$
$P(\text{crossover}) = 0,80$
$z = (p_1[0:k], p_2[k:48])$
$P(z=p_1) = 0,20$

3.5.5 Mutasi

Mutasi diterapkan secara independen pada gen di luar mandatory absolute dengan probabilitas 0,10. Pada slot 24–33, nilai baru dipilih dari tidur utama, nap, dan istirahat dengan peluang 0,05; 0,10; dan 0,85. Di luar jendela tersebut, pilihan hanya tidur utama atau istirahat dengan peluang 0,06 dan 0,94. Gen wajib tidak dimutasi. Setiap anak kemudian diperbaiki sebelum dievaluasi kembali.

$$\text{jika } rand(0,1) < p_m, \text{ maka } g_i \text{ dimutasi} \quad (3.7)$$

Keterangan:

$rand(0,1)$: bilangan acak pada rentang 0 sampai 1;
 p_m : probabilitas mutasi;
 g_i : gen pada slot ke-i;
 mutasi = perubahan nilai gen menjadi aktivitas non-wajib yang diizinkan.

Setiap gen di luar mandatory absolute diuji untuk mutasi secara independen. Distribusi nilai pengganti berbeda antara jendela nap dan slot lainnya. Gen yang termasuk mandatory absolute tidak diubah oleh mutasi.

$P(\text{mutation pada gen bebas}) = 0,10$
$p_s = (0,05; 0,10; 0,85)$ untuk $24 \leq s < 34$

$p_s=(0,06;0,94)$ untuk $s \notin [24,33]$
--

3.5.6 Penyempurnaan Solusi Menggunakan Hill Climbing

Tahap terakhir merupakan proses penyempurnaan solusi menggunakan Hill Climbing. Berbeda dengan Genetic Algorithm yang bekerja pada sekumpulan populasi, Hill Climbing hanya memproses satu solusi terbaik hasil evolusi. Sistem membangkitkan beberapa solusi tetangga (neighbor) dengan melakukan perubahan kecil pada slot aktivitas yang masih dapat dimodifikasi.

Setiap solusi tetangga kemudian dievaluasi menggunakan fungsi fitness. Apabila ditemukan solusi yang memiliki nilai fitness lebih tinggi dibandingkan solusi saat ini, maka solusi tersebut dijadikan solusi baru dan proses pencarian dilanjutkan dari posisi tersebut. Sebaliknya, apabila tidak ditemukan perbaikan setelah mencapai batas iterasi yang ditentukan, maka proses Hill Climbing dihentikan.

Melalui mekanisme tersebut, solusi terbaik hasil Genetic Algorithm dapat disempurnakan sehingga menghasilkan jadwal tidur dan aktivitas yang lebih optimal tanpa harus melakukan proses evolusi populasi Kembali

Hill Climbing memeriksa pertukaran gen bebas bersebelahan dan penggantian satu gen bebas dengan aktivitas alternatif. Setiap kandidat menjalani repair dan evaluasi fitness yang sama. Solusi kandidat diterima hanya jika skornya lebih tinggi; iterasi dihentikan ketika tidak ada perbaikan atau batas iterasi tercapai.

$Fitness(x') > Fitness(x) \Rightarrow x \leftarrow x'$
--

berhenti jika tidak ada perbaikan dalam satu iterasi atau $i = 20$
--

3.5.7 Hasil Eksekusi Genetic Algorithm dan Hill Climbing

Setelah seluruh tahapan selesai dilaksanakan, sistem menghasilkan satu kromosom terbaik sebagai solusi optimasi harian. Solusi tersebut memuat jadwal aktivitas selama satu hari yang telah memenuhi seluruh kendala wajib serta memiliki nilai fitness tertinggi di antara seluruh kandidat yang dihasilkan selama proses optimasi.

Kromosom terbaik selanjutnya digunakan sebagai masukan pada proses optimasi hari berikutnya dalam penyusunan jadwal mingguan. Informasi mengenai tingkat kelelahan akhir dan waktu mulai tidur utama juga diteruskan sebagai kondisi awal (carry-over) sehingga sistem dapat menjaga konsistensi jadwal tidur antarhari dan menghasilkan pola tidur yang lebih stabil selama periode optimasi tujuh hari.

Repair dijalankan secara berurutan. Tahap awal memulihkan aktivitas kerja, kuliah, dan transisi; tahap berikutnya membersihkan fragmen tidur, memperbaiki transisi nap, mengejar target, menjembatani celah kecil, dan membatasi nap. Kromosom yang telah diperbaiki dievaluasi kembali.

$x_{\text{repair}} = R_6 \circ R_5 \circ R_4 \circ R_3 \circ R_2 \circ R_1(x)$
--

$\text{len}(b_{\text{sleep}}) < 3 \Rightarrow \forall s \in b_{\text{sleep}}, x_s \leftarrow 3$

$g_{\text{sleep}} \leq 6 \Rightarrow \forall s \in g_{\text{sleep}}, x_s \leftarrow 0$
--

$g_{\text{nap}} \leq 2 \Rightarrow \forall s \in g_{\text{nap}}, x_s \leftarrow 2$
--

$24 \leq s_{\text{nap}} < 34 \wedge \text{len}(\text{run}_{\text{nap}}) \leq 3$

3.6 Optimasi Jadwal Tidur Mingguan

Optimasi jadwal tidur mingguan merupakan tahapan yang bertujuan menghasilkan jadwal tidur yang tidak hanya optimal pada satu hari tertentu, tetapi juga mempertahankan kualitas dan konsistensi pola tidur selama tujuh hari berturut-turut. Berbeda dengan optimasi harian yang memperlakukan setiap hari secara independen, optimasi mingguan mempertimbangkan keterkaitan kondisi fisiologis antarhari sehingga solusi yang dihasilkan lebih mendekati kondisi nyata.

Pada penelitian ini, proses optimasi dilakukan secara berurutan (sequential optimization). Setiap hari dioptimalkan satu per satu menggunakan Genetic Algorithm dan Hill Climbing dengan tetap memperhatikan kondisi yang dihasilkan pada hari sebelumnya. Pendekatan ini dipilih karena kualitas tidur pada suatu hari akan memengaruhi kondisi tubuh dan kebutuhan pemulihan pada hari berikutnya. Dengan demikian, sistem tidak hanya mencari solusi terbaik untuk setiap hari secara terpisah, tetapi juga menjaga kesinambungan jadwal tidur selama satu minggu.

Alur optimasi mingguan dimulai dari hari pertama menggunakan kondisi awal yang telah ditentukan berdasarkan data masukan pengguna. Setelah proses optimasi selesai, sistem menyimpan informasi penting yang akan diteruskan ke proses optimasi hari berikutnya. Proses tersebut dilakukan secara berulang hingga seluruh jadwal selama tujuh hari berhasil dibentuk.

Optimasi dilakukan berurutan per hari, bukan sebagai satu kromosom gabungan mingguan. Fatigue akhir, waktu mulai tidur utama, dan potongan akhir jadwal digunakan untuk membentuk konteks hari berikutnya. Senin dimulai tanpa fatigue awal, anchor, atau tail memory.

$f_{0,t}=f_{end,t-1}$
$a_t=core_start_t-1$
$tail_t=x_t-1[32:48]$
$G=200$ generasi/hari; $I_HC \leq 20$

Runner menjalankan Genetic Algorithm selama 200 generasi per hari. Optimizer menerapkan Hill Climbing pada solusi terbaik di akhir GA, dan runner mingguan memanggil Hill Climbing lagi pada kromosom yang dikembalikan. Karena itu, implementasi menjalankan penyempurnaan lokal internal dan pemanggilan lanjutan pada tahap mingguan.

3.6.1 Optimasi Hari Pertama

Proses optimasi diawali pada hari pertama menggunakan data aktivitas yang telah melalui tahap pra-pemrosesan. Pada tahap ini belum tersedia informasi mengenai kondisi hari sebelumnya sehingga sistem menggunakan nilai awal (initial state) yang telah ditentukan sebagai dasar proses optimasi.

Genetic Algorithm dan Hill Climbing kemudian dijalankan untuk menghasilkan jadwal tidur terbaik pada hari pertama. Hasil optimasi berupa kromosom dengan nilai fitness tertinggi beserta informasi pendukung, seperti waktu mulai tidur utama, durasi tidur, serta tingkat kelelahan pada akhir hari. Informasi tersebut disimpan sebagai kondisi awal untuk proses optimasi pada hari berikutnya.

3.6.2 Carry-Over Kondisi Antarhari

Setelah optimasi suatu hari selesai dilakukan, sistem tidak menghapus seluruh informasi yang dihasilkan. Beberapa kondisi penting diteruskan (carry-over) ke hari berikutnya agar proses optimasi mampu mempertahankan kesinambungan pola tidur.

Informasi yang diteruskan meliputi:
tingkat kelelahan pada akhir hari (ending fatigue);
waktu mulai tidur utama sebagai acuan regularitas tidur;
informasi kontinuitas blok tidur yang melewati batas pergantian hari (tail memory);
parameter lain yang diperlukan untuk menjaga konsistensi proses evaluasi.

Mekanisme carry-over memungkinkan sistem menyesuaikan kebutuhan pemulihan berdasarkan kondisi aktual pengguna sehingga jadwal yang dihasilkan tidak bersifat independen antarhari.

3.6.3 Optimasi Hari Kedua hingga Hari Ketujuh

Tahap berikutnya dilakukan dengan prosedur yang sama seperti pada hari pertama. Perbedaannya terletak pada kondisi awal yang digunakan. Apabila pada hari pertama sistem menggunakan kondisi awal standar, maka pada hari kedua hingga hari ketujuh kondisi awal diperoleh dari hasil optimasi hari sebelumnya.

Pada setiap hari, sistem menjalankan seluruh tahapan Genetic Algorithm dan Hill Climbing mulai dari pembentukan populasi, evaluasi fitness, proses evolusi menggunakan Genetic Algorithm, hingga penyempurnaan menggunakan Hill Climbing. Setelah solusi terbaik diperoleh, kondisi akhir kembali disimpan dan diteruskan ke hari selanjutnya.

Proses ini berlangsung secara berulang hingga seluruh jadwal selama tujuh hari berhasil dioptimalkan.

3.6.4 Penyusunan Jadwal Mingguan

Setelah seluruh proses optimasi harian selesai dilakukan, sistem menggabungkan hasil optimasi dari masing-masing hari menjadi satu jadwal mingguan yang utuh.

Penyusunan jadwal dilakukan dengan mempertahankan urutan kronologis setiap hari sehingga pengguna dapat melihat distribusi aktivitas dan waktu tidur selama satu minggu secara keseluruhan. Selain menghasilkan jadwal lengkap, sistem juga menghitung nilai evaluasi mingguan yang diperoleh dari akumulasi hasil optimasi setiap hari.

Informasi yang disajikan meliputi:

1. jadwal aktivitas harian selama tujuh hari;
2. durasi tidur setiap hari;
3. waktu mulai tidur utama setiap hari;
4. tingkat kelelahan akhir setiap hari;
5. nilai fitness harian;
6. nilai fitness total selama satu minggu.

Penyajian informasi tersebut memudahkan proses analisis terhadap kualitas jadwal yang dihasilkan serta mempermudah proses perbandingan dengan metode pembandingan pada tahap evaluasi.

3.6.5 Hasil Optimasi Mingguan

Tahap akhir menghasilkan satu paket jadwal tidur mingguan yang telah memenuhi seluruh kendala aktivitas wajib serta dioptimalkan berdasarkan kualitas tidur, durasi tidur, tingkat kelelahan, dan konsistensi pola tidur antarminggu.

Hasil optimasi ini selanjutnya digunakan sebagai masukan pada tahap Baseline Comparison dan Evaluasi Sistem untuk mengetahui efektivitas Genetic Algorithm dan Hill Climbing dibandingkan metode pembandingan. Dengan pendekatan optimasi secara berurutan dan mekanisme carry-over, sistem diharapkan mampu menghasilkan jadwal tidur yang lebih realistis, adaptif, dan konsisten dibandingkan apabila setiap hari dioptimalkan secara terpisah.

3.7 Konfigurasi Parameter Eksperimen Utama

Eksperimen final dijalankan dengan population size 50, mutation rate 0,10, crossover rate 0,80, dan random seed 42. Optimasi mingguan menggunakan 200 generasi per hari dan Hill Climbing dengan batas 20 iterasi. Konfigurasi tersebut tercatat pada METADATA.json sebagai parameter default_fallback.

Kalibrasi parameter tidak dijalankan ulang pada eksekusi final karena pipeline dipanggil dengan opsi --skip-calibration. Dengan demikian, penelitian ini melaporkan parameter yang benar-benar digunakan dan tidak menyatakan bahwa hasil final berasal dari konfigurasi terbaik hasil grid search.

Pemisahan dataset kalibrasi dan pengujian tetap dicatat pada struktur proyek, tetapi hanya dataset pengujian final berisi 63 responden yang menjadi dasar angka evaluasi pada Bab IV. Seed tetap 42 digunakan untuk mengendalikan pembangkitan acak pada populasi dan operator GA.

3.8 Baseline Comparison

Baseline comparison digunakan untuk menilai kinerja Genetic Algorithm dan Hill Climbing secara komparatif terhadap dua metode pembandingan yang lebih sederhana. Perbandingan dilakukan pada hari Rabu sebagai hari representatif pertengahan minggu. Metrik yang dibandingkan adalah nilai fitness, kelelahan akhir hari (fatigue), dan durasi tidur (sleep_hours).

Pemilihan kedua baseline tersebut bertujuan memberikan dua titik pembandingan yang berbeda. Greedy schedule merepresentasikan pendekatan adaptif sederhana yang hanya mempertimbangkan slot bebas terpanjang, sedangkan fixed schedule merepresentasikan pola tidur konvensional yang bersifat tetap dan tidak menyesuaikan diri terhadap variasi aktivitas harian.

3.8.1 Greedy Schedule

Greedy schedule membentuk jadwal tidur dengan strategi satu langkah tanpa optimasi lanjutan. Algoritma mengidentifikasi seluruh slot bebas di luar mandatory absolute, mengelompokkannya menjadi blok-blok berurutan, mengurutkan blok tersebut dari yang terpanjang hingga terpendek, kemudian mengisi blok-blok tersebut dengan tidur secara berurutan hingga target 14 slot terpenuhi. Apabila target sudah tercapai, slot bebas yang

tersisa dibiarkan sebagai aktivitas santai. Greedy schedule tidak melakukan evaluasi fitness maupun perbaikan kualitas tidur; ia semata-mata mengisi ruang terpanjang yang tersedia.

3.8.2 Fixed Schedule

Fixed schedule membentuk jadwal tidur tetap yang diasumsikan dimulai pukul 22.00 dengan durasi 7 jam, setara dengan 14 slot mulai dari slot ke-44. Apabila slot-slot tidur tersebut bertabrakan dengan mandatory absolute, aktivitas wajib tetap dipertahankan dan slot bersangkutan tidak diberi kode tidur. Fixed schedule merepresentasikan pola tidur konvensional seseorang yang tidak menyesuaikan jadwal tidurnya dengan variasi aktivitas harian.

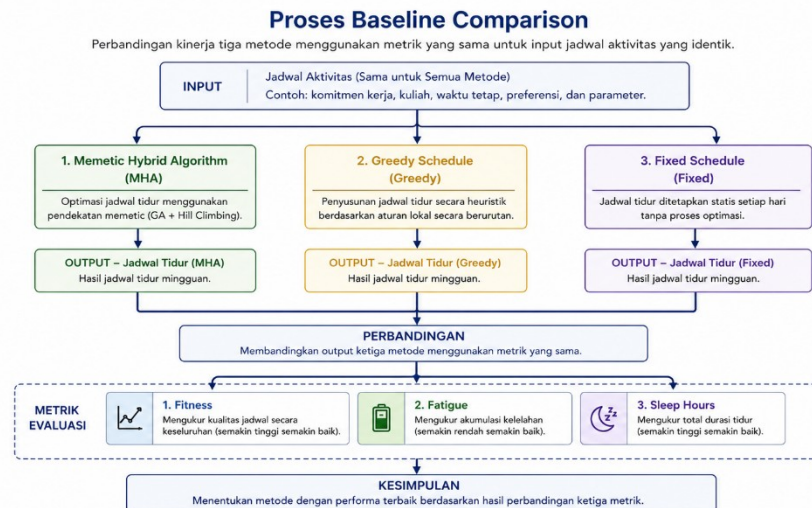
3.8.3 Evaluasi Komparatif

Ketiga metode—Genetic Algorithm dan Hill Climbing, greedy schedule, dan fixed schedule—dievaluasi menggunakan fungsi fitness yang sama, sehingga perbandingan metrik dapat dilakukan secara konsisten dan adil. Hasil dari setiap metode meliputi nilai fitness, fatigue akhir hari, dan durasi tidur dalam jam. Selisih nilai fitness, fatigue, dan durasi tidur antara Genetic Algorithm dan Hill Climbing dengan kedua baseline menjadi indikator utama apakah pendekatan yang diusulkan menghasilkan jadwal yang lebih baik.

$$Improvement(i) = \frac{Fitness_{Sora} - Fitness_{Baseline}}{|Fitness_{Baseline}|} \times 100 \quad (3.9)$$

Keterangan :

$Improvement(i)$	: persentase peningkatan fitness;
$Fitness_{Sora}$	: fitness metode Genetic Algorithm dan Hill Climbing;
$Fitness_{Baseline}$	: fitness metode pembandingan;
$ Fitness_{Baseline} $	: nilai absolut fitness baseline.



Gambar 3. 6 Baseline Comparison dan Evaluasi

Sumber: Diolah penulis

3.9 Evaluasi Sistem

Evaluasi sistem dilakukan untuk mengukur kinerja Genetic Algorithm dan Hill Climbing dalam menghasilkan jadwal tidur yang optimal berdasarkan data aktivitas pengguna. Tahap evaluasi bertujuan untuk mengetahui apakah algoritma yang diusulkan mampu menghasilkan solusi yang lebih baik dibandingkan metode pembanding serta mampu mempertahankan performa yang konsisten pada berbagai karakteristik pengguna. Eksperimen final menggunakan konfigurasi default_fallback yang tercatat pada METADATA.json; kalibrasi dilewati dengan opsi --skip-calibration. Karena itu hasil evaluasi tidak diklaim sebagai performa dari parameter terbaik hasil grid search.

Pada penelitian ini, evaluasi sistem dilaksanakan melalui tiga skenario pengujian, yaitu Weekly Optimization Test, Baseline Comparison, dan Multi-Subject Test. Ketiga skenario tersebut dirancang untuk mengevaluasi performa algoritma dari aspek kualitas solusi, kemampuan adaptasi terhadap perubahan aktivitas, serta konsistensi hasil optimasi pada berbagai kondisi pengguna.

3.9.1 Weekly Optimization Test

Pengujian pertama dilakukan untuk mengevaluasi kemampuan sistem dalam menghasilkan jadwal tidur selama tujuh hari secara berurutan. Pengujian ini bertujuan untuk mengetahui apakah mekanisme optimasi mingguan mampu mempertahankan kualitas jadwal tidur tanpa menyebabkan penurunan performa pada hari-hari berikutnya.

Pada tahap ini, sistem menerima data aktivitas selama satu minggu yang telah melalui proses pra-pemrosesan. Selanjutnya Genetic Algorithm dan Hill Climbing dijalankan secara berurutan mulai dari hari pertama hingga hari ketujuh dengan menerapkan mekanisme carry-over antarhari. Setelah seluruh proses optimasi selesai, sistem menghasilkan satu jadwal mingguan beserta nilai evaluasi setiap hari.

Beberapa informasi yang diamati selama pengujian meliputi:

1. nilai fitness harian;

2. nilai fitness total selama satu minggu;
3. durasi tidur setiap hari;
4. waktu mulai tidur utama;
5. tingkat kelelahan pada akhir setiap hari;
6. konsistensi pola tidur antarhari.

Hasil pengujian ini digunakan untuk mengetahui kemampuan algoritma dalam menjaga kualitas jadwal tidur selama periode optimasi tanpa mengoptimalkan setiap hari secara terpisah.

3.8 Baseline Comparison

Pengujian kedua dilakukan dengan membandingkan performa Genetic Algorithm dan Hill Climbing terhadap dua metode pembandingan (baseline), yaitu Greedy Schedule dan Fixed Schedule. Pengujian ini bertujuan untuk mengetahui sejauh mana algoritma yang diusulkan mampu menghasilkan solusi yang lebih baik dibandingkan pendekatan sederhana.

Pada setiap skenario pengujian, ketiga metode menggunakan data masukan yang sama sehingga seluruh metode menghadapi kondisi aktivitas dan kendala yang identik. Setelah proses optimasi selesai, hasil dari masing-masing metode dibandingkan berdasarkan indikator evaluasi yang telah ditentukan.

Aspek yang dibandingkan meliputi:

- II. nilai fitness akhir;
- III. durasi tidur yang diperoleh;
- IV. jumlah pelanggaran terhadap aturan penjadwalan;
- V. tingkat kelelahan akhir;
- VI. konsistensi waktu mulai tidur selama satu minggu.

Perbandingan tersebut digunakan untuk menunjukkan peningkatan performa yang diperoleh melalui penerapan Genetic Algorithm dan Hill Climbing dibandingkan metode yang tidak menggunakan mekanisme optimasi evolusioner maupun pencarian lokal.

3.9.3 Multi-Subject Test

Selain dilakukan pada satu data pengguna, sistem juga diuji menggunakan beberapa data subjek yang memiliki karakteristik aktivitas berbeda. Pengujian ini bertujuan untuk mengetahui kemampuan generalisasi algoritma terhadap variasi kondisi pengguna.

Setiap subjek memiliki kombinasi jadwal kerja, jadwal kuliah, waktu luang, dan kepadatan aktivitas yang berbeda-beda. Seluruh data diproses menggunakan prosedur optimasi yang sama tanpa melakukan perubahan terhadap parameter algoritma. Dengan demikian, perbedaan hasil yang diperoleh sepenuhnya dipengaruhi oleh karakteristik aktivitas masing-masing subjek.

Hasil optimasi setiap subjek kemudian dianalisis untuk mengetahui apakah algoritma tetap mampu menghasilkan jadwal tidur yang layak dan berkualitas pada berbagai kondisi aktivitas.

3.9.4 Indikator Evaluasi

Setelah seluruh skenario pengujian selesai dilaksanakan, hasil optimasi dianalisis menggunakan beberapa indikator evaluasi yang digunakan sebagai ukuran performa sistem.

Indikator yang digunakan pada penelitian ini meliputi:

- I. Nilai Fitness, yaitu indikator utama yang menunjukkan kualitas keseluruhan jadwal hasil optimasi.
- II. Durasi Tidur Harian, yaitu total waktu tidur utama yang diperoleh pada setiap hari selama periode optimasi.
- III. Tingkat Kelelahan Akhir (Ending Fatigue), yaitu kondisi kelelahan pengguna setelah seluruh aktivitas harian selesai dijalankan.
- IV. Regularitas Tidur, yaitu konsistensi waktu mulai tidur utama antarhari yang dihasilkan selama optimasi mingguan.
- V. Jumlah Pelanggaran Kendala, yaitu banyaknya pelanggaran terhadap aturan penjadwalan yang masih ditemukan setelah proses optimasi selesai.

Seluruh indikator tersebut dianalisis secara bersamaan sehingga evaluasi tidak hanya berfokus pada nilai fitness, tetapi juga mempertimbangkan kualitas jadwal tidur yang dihasilkan dari sisi biologis maupun operasional.

Keluaran dievaluasi menggunakan fitness, fatigue akhir, durasi tidur, recovery, penalti, waktu mulai blok tidur utama, sesi nap, perubahan terhadap baseline observasi, dan selisih fitness terhadap baseline.

$\text{sleep_hours}=0,5A$
$\text{fitness_delta}=\text{Fitness_opt}-\text{Fitness_obs}$
$\text{sleep_change_slots}= S_opt\Delta S_obs $
$\text{fatigue_end}=f_{47}$

3.9.5 Analisis Hasil Pengujian

Tahap terakhir merupakan analisis terhadap seluruh hasil pengujian yang telah diperoleh. Analisis dilakukan dengan membandingkan performa Genetic Algorithm dan Hill Climbing pada ketiga skenario pengujian untuk mengetahui tingkat efektivitas algoritma dalam menyelesaikan permasalahan optimasi jadwal tidur.

Hasil pengujian disajikan dalam bentuk tabel dan visualisasi yang memuat nilai setiap indikator evaluasi. Selanjutnya dilakukan pembahasan mengenai kelebihan dan keterbatasan algoritma berdasarkan hasil yang diperoleh selama proses pengujian.

Melalui tahapan evaluasi tersebut, penelitian ini dapat memberikan gambaran mengenai kemampuan Genetic Algorithm dan Hill Climbing dalam menghasilkan jadwal tidur yang memenuhi kendala aktivitas, menjaga konsistensi pola tidur antarhari, serta memberikan kualitas solusi yang lebih baik dibandingkan metode pembanding. Dengan demikian, hasil evaluasi menjadi dasar dalam menjawab tujuan penelitian sekaligus menilai keberhasilan implementasi sistem optimasi yang dikembangkan.

3.10 Implementasi Program

Sistem dikembangkan menggunakan bahasa pemrograman Python dengan pustaka inti optimasi pada paket src/sora, layanan REST API berbasis FastAPI (api.py), serta

antarmuka web berbasis React dan Streamlit sehingga pengguna dapat melakukan proses optimasi jadwal tidur melalui peramban web secara interaktif.

Implementasi sistem dibagi menjadi beberapa modul utama agar proses pengembangan, pengujian, dan pemeliharaan program dapat dilakukan secara terstruktur. Setiap modul memiliki tanggung jawab yang berbeda mulai dari pembacaan data, proses optimasi, evaluasi hasil, hingga penyajian laporan kepada pengguna.

3.10.1 Arsitektur Program

Secara umum, sistem terdiri atas empat kelompok utama, yaitu pustaka inti optimasi, layanan API, antarmuka pengguna, dan skrip eksperimen.

Pustaka inti optimasi (`src/sora`) memuat seluruh komponen komputasi, mulai dari pembacaan data (`data/parser.py`), engine Genetic Algorithm (`genetic_algorithm/`), Hill Climbing (`hill_climbing/`), optimasi mingguan (`experiments/weekly.py`), kalibrasi parameter (`experiments/calibration.py`), metode baseline (`experiments/baseline.py`), pengujian multi-subjek (`experiments/multi_subject.py`), hingga indikator evaluasi (`utils/metrics.py`).

Layanan API (`api.py`) menyediakan REST API berbasis FastAPI yang menjembatani antarmuka pengguna dengan pustaka inti, meliputi unggah data, menjalankan optimasi, dan mengambil hasil.

Antarmuka pengguna terdiri atas aplikasi web berbasis React yang dibangun menggunakan Vite (`frontend/src`) serta aplikasi Streamlit (`frontend/app.py`). Keduanya menangani input data, menjalankan optimasi, menampilkan jadwal, dan mengeksport laporan.

Skrip eksperimen pada folder `scripts` (`demo_thesis.py` dan `generate_report.py`) digunakan untuk menjalankan demonstrasi dan menyusun laporan pengujian secara otomatis. Pemisahan kelompok tersebut memudahkan pengembangan karena perubahan pada satu bagian program tidak langsung memengaruhi bagian lainnya.

3.10.2 Modul Pembacaan dan Pra-Pemrosesan Data

Modul pertama bertugas membaca data aktivitas pengguna dari berkas CSV yang diunggah melalui antarmuka aplikasi. Setelah data dibaca, sistem mengubah seluruh informasi waktu ke dalam representasi 48 slot per hari dengan interval 30 menit.

Pada tahap ini sistem juga:

- membentuk mandatory core untuk aktivitas wajib;
- membentuk transit slot antaraktivitas;
- membentuk mandatory absolute yang tidak boleh diubah oleh algoritma;
- melakukan feasibility check untuk mendeteksi konflik jadwal.

Apabila ditemukan konflik, sistem menampilkan pesan kesalahan dan proses optimasi tidak dijalankan sampai data diperbaiki.

3.10.3 Modul Optimasi Harian

Modul optimasi harian merupakan inti proses komputasi. Modul ini menerima kromosom hasil pra-pemrosesan kemudian menjalankan seluruh tahapan Genetic Algorithm dan Hill Climbing.

Proses yang dilakukan meliputi:

- pembentukan populasi awal;

evaluasi fitness;
seleksi dan elitisme;
crossover;
mutasi;
repair chromosome;
penyempurnaan solusi menggunakan Hill Climbing.

Hasil akhir modul ini adalah satu jadwal harian terbaik beserta nilai fitness dan informasi evaluasi pendukung.

3.10.4 Modul Optimasi Mingguan

Modul optimasi mingguan menjalankan optimasi secara berurutan selama tujuh hari. Setelah satu hari selesai dioptimalkan, sistem menyimpan kondisi akhir hari tersebut dan menggunakannya sebagai kondisi awal pada hari berikutnya.

Informasi yang diteruskan antarmuka meliputi:
tingkat kelelahan akhir;
waktu mulai tidur utama;
kontinuitas blok tidur yang melewati pergantian hari.

Dengan mekanisme tersebut, jadwal mingguan yang dihasilkan menjadi lebih konsisten dan tidak memperlakukan setiap hari sebagai masalah yang sepenuhnya terpisah.

3.10.5 Modul Evaluasi dan Baseline Comparison

Setelah optimasi selesai dilakukan, sistem menjalankan modul evaluasi untuk menghitung seluruh indikator pengujian. Modul ini juga menjalankan dua metode pembandingan, yaitu Greedy Schedule dan Fixed Schedule.

Hasil dari ketiga metode disimpan dalam struktur data yang sama sehingga dapat dibandingkan secara langsung berdasarkan:
nilai fitness;
durasi tidur;
tingkat kelelahan akhir;
konsistensi waktu mulai tidur;
jumlah pelanggaran kendala.

3.10.6 Modul Antarmuka Streamlit

Antarmuka pengguna disediakan dalam dua bentuk, yaitu aplikasi web berbasis React yang dibangun menggunakan Vite (frontend/src) dan aplikasi Streamlit (frontend/app.py). Melalui kedua antarmuka ini pengguna dapat:
mengunggah berkas CSV aktivitas;
memilih parameter optimasi;
menjalankan proses optimasi;
melihat jadwal harian dan mingguan;
membandingkan hasil dengan metode baseline;

mengunduh laporan hasil evaluasi.

Seluruh proses komputasi dijalankan di sisi server melalui layanan API (api.py), sedangkan hasilnya ditampilkan secara otomatis pada antarmuka web.

3.10.7 Struktur Implementasi Program

Untuk memudahkan pengelolaan kode, program diorganisasikan ke dalam beberapa berkas dan folder utama. Struktur implementasi yang digunakan ditunjukkan pada Tabel 3.3.

Modul	Fungsi
app.py	Mengelola antarmuka aplikasi Streamlit dan alur utama aplikasi.
api.py	Menyediakan REST API berbasis FastAPI untuk layanan antarmuka web.
src/sora/data/parser.py	Membaca dan memvalidasi data aktivitas dari berkas CSV.
src/sora/genetic_algorithm/ (9 modul)	Menjalankan proses Genetic Algorithm (representasi, populasi, fitness, seleksi, crossover, mutasi, dan repair).
src/sora/hill_climbing/local_search.py	Melakukan penyempurnaan solusi lokal.
src/sora/experiments/weekly.py	Mengelola optimasi tujuh hari dan carry-over antarhari.
src/sora/experiments/calibration.py	Menjalankan kalibrasi parameter dengan Grid Search.
src/sora/experiments/baseline.py	Menjalankan Greedy dan Fixed Schedule.
src/sora/experiments/multi_subject.py	Menjalankan pengujian multi-subjek.
src/sora/utlis/metrics.py	Menghitung indikator evaluasi kinerja.
scripts/demo_thesis.py dan generate_report.py	Menjalankan demonstrasi dan menyusun laporan hasil pengujian.

3.10.8 Hasil Implementasi

Melalui implementasi tersebut, sistem mampu menerima data aktivitas pengguna, menjalankan proses optimasi jadwal tidur selama tujuh hari, membandingkan hasil dengan metode baseline, serta menampilkan laporan evaluasi secara otomatis melalui antarmuka web.

Hasil implementasi inilah yang selanjutnya digunakan pada Bab IV untuk melakukan pengujian dan pembahasan terhadap performa Genetic Algorithm dan Hill Climbing dalam menghasilkan jadwal tidur yang optimal pada pekerja dengan pola aktivitas tidak teratur.

BAB IV

HASIL PENGUJIAN DAN PEMBAHASAN

Bab ini menyajikan hasil pengujian sistem SORA (Sleep Optimization and Rest Arrangement) pada data final, kemudian menjelaskan makna dan batas interpretasi setiap metrik. Dataset pengujian memuat 63 baris dengan 63 responden unik. Setiap responden diproses untuk tujuh hari secara berurutan, sehingga terdapat 441 evaluasi harian. Jadwal kerja dan kuliah diperlakukan sebagai aktivitas wajib, sedangkan jadwal tidur dari kuesioner menjadi baseline observasi. Semua angka pada bab ini merupakan hasil evaluasi komputasional dengan seed 42; skor fitness adalah skor fungsi tujuan yang tidak memiliki satuan dan bukan persentase akurasi atau ukuran klinis.

4.1 Implementasi Sistem

Implementasi penelitian menghasilkan SORA Academic, yaitu aplikasi optimasi jadwal tidur yang terdiri atas mesin optimasi berbasis Python, layanan REST API menggunakan FastAPI, dan antarmuka web React. Mesin optimasi merealisasikan rancangan pada Bab III menjadi alur yang dapat menerima data aktivitas, membentuk jadwal harian, menjalankan optimasi, dan mengembalikan jadwal beserta metriknya. Antarmuka dan API menghubungkan fungsi tersebut dengan pengguna. Implementasi ini merupakan prototipe penelitian yang dijalankan pada lingkungan lokal; hasilnya tidak menyatakan bahwa sistem telah dipasang sebagai layanan produksi atau digunakan untuk memberi rekomendasi klinis.

4.1.1 Implementasi Dataset dan Normalisasi

Parser membaca dataset kuesioner final kuisoner (Jawaban) - Form Responses 1.csv. Kolom nama sumber dan kolom aktivitas untuk tujuh hari dipetakan menjadi Nama, Kerja_<Hari>, Kuliah_<Hari>, dan Tidur_<Hari>. Nilai kosong, nan, null, none, atau tanda hubung untuk rentang aktivitas menghasilkan daftar slot kosong. Rentang waktu yang dipisahkan koma dibaca satu per satu, divalidasi dengan pola jam-menit, lalu digabungkan sebagai daftar indeks slot unik yang diurutkan. Parser juga menangani rentang yang melewati tengah malam dengan melanjutkan indeks ke awal hari berikutnya.

Contoh input anonim dari format kuesioner: 07:30 - 12:30. Keluaran parser adalah indeks slot 15 sampai 24, yaitu sepuluh slot aktivitas yang masing-masing berdurasi 30 menit. Pada struktur data normalisasi, contoh tersebut disimpan pada kolom Kerja_<Hari> sebagai teks daftar indeks; identitas responden tidak diperlukan dalam contoh ini. Jika format waktu tidak cocok atau indeks slot terduplikasi/tidak berada pada rentang yang diizinkan, proses menghasilkan kesalahan validasi alih-alih meneruskan nilai yang tidak sah.

4.1.2 Representasi Kromosom dan Constraint

Satu kromosom harian memuat 48 gen, satu gen untuk setiap slot 30 menit. Enumerasi Allele pada representation.py menetapkan SLEEP=0, WORK=1, NAP=2, REST=3, dan CLASS=4. Contoh representasi: jika aktivitas kerja berlangsung pada slot 15–24, maka gen pada indeks tersebut bernilai WORK (1); nilai pada indeks lain mengikuti aktivitas dalam kandidat jadwal. Fungsi validate_chromosome memeriksa panjang kromosom dan menolak alel di luar lima nilai tersebut. Jadwal kerja dan kuliah membentuk mandatory_core; satu slot di sekitar setiap slot wajib yang tidak bersebelahan ditambahkan

sebagai transit slot. Gabungan keduanya menjadi `mandatory_absolute` yang tidak boleh diubah operator optimasi.

Pemeriksaan constraint juga memvalidasi bahwa daftar slot merupakan bilangan bulat dalam rentang 0–47, tidak memiliki duplikasi, dan slot kerja tidak tumpang tindih dengan slot kuliah. Hari dengan lebih dari 36 slot wajib tidak dijalankan oleh optimasi mingguan dan diberi status `skipped_infeasible`. Jika ruang kosong hari tersebut tidak memiliki rentang kontinu sedikitnya 12 slot, konteks ditandai `survival_mode`; aturan fitness dan repair menggunakan perlakuan khusus untuk kondisi jadwal padat ini.

4.1.3 Implementasi Fungsi Fitness

Fungsi `calculate_fitness` pada `fitness.py` mengembalikan fitness, fatigue akhir, durasi tidur, awal blok tidur utama, `recovery_reward`, `dynamic_penalty`, `regularity_penalty`, dan jumlah sesi nap. `Recovery` dihitung sepanjang slot berdasarkan aktivitas dan bobot waktu sirkadian; kerja, kuliah, dan istirahat menambah fatigue dengan laju berbeda, sedangkan tidur dan nap yang memenuhi aturan mengurangnya serta dapat memberi reward. Penalti dinamis memeriksa durasi tidur terhadap minimum, maksimum, dan target; kesesuaian dengan durasi observasi; tumpang tindih slot tidur; jumlah blok; jumlah serta waktu nap. Penalti regularitas membandingkan awal tidur utama dengan anchor hari sebelumnya dan dibatasi maksimum 400. Skor akhir mengikuti ekspresi pada Tabel 4.1; fitness adalah skor internal fungsi tujuan, bukan akurasi maupun ukuran klinis.

Tabel 4.1 Persamaan Fitness yang Diterapkan dalam Implementasi

$F = w_s \times (R + 0,2D) + P_d + P_r$

Keterangan: w_s adalah bobot tidur; R recovery reward; D jumlah gen tidur (slot); P_d dynamic penalty; dan P_r regularity penalty. Persamaan ini setara dengan ekspresi yang dieksekusi pada `fitness.py`.

4.1.4 Implementasi Genetic Algorithm

Pada setiap generasi Genetic Algorithm, populasi dirangking dari fitness tertinggi. Individu elit dipertahankan, sedangkan indeks dua induk dipilih dari kelompok teratas. Dengan probabilitas crossover, satu anak dibentuk melalui single-point crossover pada titik potong yang dipilih di antara slot 10 dan 37; jika crossover tidak terjadi, kromosom induk pertama disalin. Mutasi hanya menyentuh slot nonwajib dan memakai distribusi alel yang berbeda untuk jendela nap dan slot di luar jendela nap. Anak kemudian melalui repair sebelum masuk ke generasi berikutnya. Pengujian utama menggunakan populasi 50, mutasi 0,10, crossover 0,80, 200 generasi, dan seed 42 ditambah indeks hari.

4.1.5 Implementasi Hill Climbing

Hill Climbing membentuk kandidat tetangga dari pertukaran berurutan antara alel tidur, nap, atau istirahat yang berbeda serta perubahan alel pada slot nonwajib. Setiap kandidat diperbaiki dan dinilai kembali; kandidat diterima hanya jika fitness-nya lebih tinggi daripada skor terbaik sementara. Iterasi berhenti setelah batas iterasi tercapai atau satu putaran penuh tidak menemukan perbaikan. Pada implementasi aktual, optimizer menjalankan Hill Climbing pada individu terbaik setelah GA; `weekly.py` kemudian memanggil `run_hill_climbing` sekali lagi pada hasil tersebut. Karena itu proses mingguan memiliki dua pemanggilan pencarian lokal, masing-masing dengan batas konfigurasi 20

iterasi, bukan satu pemanggilan total 20 iterasi. Perilaku ini dituliskan apa adanya agar uraian dan cuplikan kode konsisten dengan eksperimen yang menghasilkan data final.

4.1.6 Implementasi Optimasi Mingguan dan API

Fungsi `run_weekly_optimization_with_history` memproses Senin sampai Minggu secara berurutan. Untuk tiap hari fungsi membaca slot kerja, kuliah, dan tidur observasi, membentuk optimizer dengan fatigue awal, anchor awal tidur, debt installment, memori 16 slot terakhir, dan seed harian, lalu mengecek feasibility. Jika layak, proses menjalankan GA dan pencarian lokal, menghitung metrik, serta menyimpan jadwal hasil dan jadwal observasi. Nilai fatigue akhir, awal blok tidur, dan 16 gen terakhir diteruskan ke hari berikutnya. Hasil harian memuat fitness, fitness observasi, fitness_delta, sleep_hours, sleep_change_slots, recovery, penalti, regularity_penalty, nap_sessions, dan status.

Endpoint POST `/api/weekly/optimize` di `api.py` menerima nama subjek, dataset, parameter, dan seed melalui `WeeklyRequest`. Endpoint meneruskan permintaan ke fungsi optimasi mingguan dan mengembalikan `target_name`, `schedule`, `observed_schedule`, `metrics`, serta `history`. Jika subjek tidak ditemukan, endpoint mengirim respons HTTP 404. Contoh keluaran secara struktural berisi tujuh jadwal yang masing-masing mempunyai 48 alel, tujuh objek metrik harian, dan riwayat generasi; nilai contoh dalam naskah tidak menggunakan nama atau identitas responden.

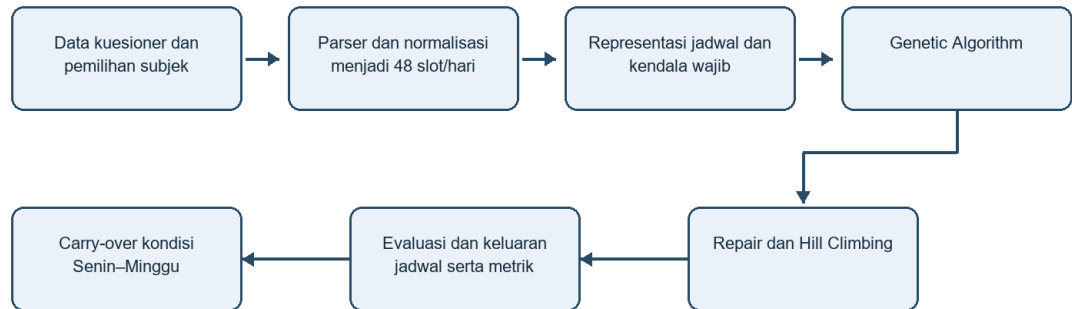
4.1.7 Implementasi Antarmuka dan Keluaran

Antarmuka React menyajikan halaman Beranda, Dataset, Mingguan, Baseline, dan Kalibrasi. Pada halaman Mingguan, pengguna memilih subjek dan menjalankan optimasi; hasilnya ditampilkan sebagai perbandingan jadwal observasi dan jadwal optimasi dalam garis waktu, disertai perubahan slot, durasi tidur, nap, fitness, serta riwayat generasi. Halaman Baseline meminta API membandingkan metode SORA, greedy, dan fixed untuk subjek terpilih. Halaman Kalibrasi menjalankan endpoint grid search dan menerima ringkasan parameter terbaik. Proyek juga menyediakan antarmuka alternatif Streamlit pada `frontend/app.py` dengan menu Dataset, Weekly, Baseline, Calibration, dan Multi-Subject. Angka pada penelitian tetap merujuk pada artefak pipeline final yang parameternya tercatat.

Peta kode inti mencakup `parser.py` untuk pembacaan dan normalisasi; `representation.py` dan `constraints.py` untuk alel, kromosom, serta kendala; `fitness.py` untuk penilaian; `population.py`, `selection.py`, `crossover.py`, `mutation.py`, `repair.py`, dan `optimizer.py` untuk Genetic Algorithm; `local_search.py` untuk Hill Climbing; `weekly.py` untuk alur tujuh hari; serta `api.py` untuk endpoint. Cuplikan kode inti pada bagian berikut diambil langsung dari berkas tersebut. Kode lengkap tetap berada di direktori proyek `sora_academic`; listing dipilih seperlunya agar tiap tahap dapat ditelusuri tanpa memenuhi badan skripsi dengan seluruh kode sumber.

Hasil implementasi pada data final mencakup pemrosesan 63 responden selama tujuh hari, 441 keluaran harian berstatus optimized, dan 63 pasangan keluaran hari Rabu untuk baseline. Status optimized menunjukkan bahwa proses komputasi menghasilkan keluaran, bukan jaminan jadwal ideal atau kelayakan klinis.

Alur implementasi SORA Academic



Gambar 4.1 Alur implementasi SORA Academic

Sumber: Diolah penulis berdasarkan implementasi SORA Academic

4.1.8 Pseudocode dan Cuplikan Kode Inti

Pseudocode 4.1 Alur pemrosesan satu subjek selama tujuh hari (pseudocode, bukan kode sumber)

BACA berkas kuesioner dan normalisasikan kolom aktivitas
PILIH satu baris subjek; inisialisasi fatigue, anchor, dan memori slot
UNTUK setiap hari Senin sampai Minggu:
 PARSE slot kerja, kuliah, dan tidur observasi
 VALIDASI irisan kerja-kuliah dan kelayakan slot wajib
 BENTUK konteks optimasi dengan seed dasar + indeks hari
 BANGUN jadwal observasi sebagai pembanding
 JIKA layak: jalankan GA selama 200 generasi dan lakukan repair
 JALANKAN Hill Climbing internal optimizer pada kandidat terbaik
 JALANKAN pemanggilan Hill Climbing eksplisit pada weekly.py
 HITUNG fitness, fatigue, recovery, penalti, dan perubahan slot
 SIMPAN jadwal; teruskan fatigue, anchor, serta 16 slot terakhir
KEMBALIKAN jadwal mingguan, metrik harian, dan riwayat konvergensi

Cuplikan kode berikut disalin langsung dari berkas sumber yang disebutkan pada judul listing. Setiap cuplikan menunjukkan operasi aktual; uraian algoritmik di atas adalah pseudocode untuk merangkum urutan proses.

Cuplikan kode 4.1 Parser rentang waktu (src/sora/data/parser.py)

```
def parse_time_ranges(value) -> list[int]:
    """Convert questionnaire time ranges into unique 30-minute slot indices."""
    text = str(value).strip()
    if text.lower() in {"nan", "", "-", "null", "none"}:
        return []
    slots: list[int] = []
    for item in text.split(","):
        match = re.fullmatch(r"\s*(\d{1,2}):(\d{2})\s*-\s*(\d{1,2}):(\d{2})\s*", item)
        if not match:
            raise ValueError(f"Invalid time range format: {item!r}")
        start_hour, start_minute, end_hour, end_minute = map(int, match.groups())
        start = start_hour * 2 + start_minute // 30
        end = end_hour * 2 + end_minute // 30
        if start < 0 or end > 48 or start_hour > 24 or end_hour > 24:
            raise ValueError(f"Time range is outside one day: {item!r}")
        if end <= start:
            end += 48
        slots.extend(slot % 48 for slot in range(start, end))
    return sorted(set(slots))
```

Cuplikan kode 4.2 Alel aktivitas dalam kromosom (src/sora/genetic_algorithm/representation.py)

```
class Allele(IntEnum):
    """Activity allele used in each 30-minute gene."""

    SLEEP = 0
    WORK = 1
    NAP = 2
    REST = 3
    CLASS = 4
```

Cuplikan kode 4.3 Perhitungan skor fitness akhir (src/sora/genetic_algorithm/fitness.py)

```
fitness = (context.sleep_weight * (recovery_reward + actual_duration * 0.2)) + dynamic_penalty +
regularity_penalty
```

Cuplikan kode 4.4 Siklus generasi dan operator GA (src/sora/genetic_algorithm/optimizer.py)

```
for generation_idx in range(generations):
    population = rank_population(population, self.calculate_fitness)
    if collect_history:
        top_results = [self.calculate_fitness(chromosome).fitness for chromosome in
population[:10]]
        history.append(
            {
                "generation": float(generation_idx + 1),
                "best_fitness": float(top_results[0]),
                "mean_top10_fitness": float(np.mean(top_results)),
                "schedule": population[0][:],
            }
        )
    elite_count = elitism_count(self.context.population_size)
    next_generation = population[:elite_count]

    while len(next_generation) < self.context.population_size:
        parent_a_idx, parent_b_idx = select_parent_indices(population, self.rng)
        if self.rng.random() < self.context.crossover_rate:
            child = single_point_crossover(population[parent_a_idx], population[parent_b_idx],
self.rng)
        else:
            child = population[parent_a_idx][:]
            child = mutate_chromosome(child, self.context, self.rng)
            child = repair_chromosome(child, self.context)
            next_generation.append(child)

    population = next_generation
    if progress_callback and (
        generation_idx + 1 == generations or (generation_idx + 1) % progress_step == 0
    ):
        progress_callback(generation_idx + 1, generations)

    population = rank_population(population, self.calculate_fitness)
```

Cuplikan kode 4.5 Urutan repair kromosom (src/sora/genetic_algorithm/repair.py)

```
def repair_chromosome(chromosome: list[int], context: ScheduleContext) -> list[int]:
    """Run all repair procedures in dependency-safe order."""
    validate_chromosome(chromosome)
    chromosome = enforce_mandatory_slots(chromosome, context)
    chromosome = remove_micro_sleeps(chromosome, context)
    chromosome = sanitize_transitions(chromosome, context)
    chromosome = enforce_sleep_target(chromosome, context)
    chromosome = bridge_gaps(chromosome, context)
    chromosome = limit_nap_duration(chromosome, context)
    return chromosome
```

Cuplikan kode 4.6 Pencarian lokal Hill Climbing (src/sora/hill_climbing/local_search.py)

```
def run_hill_climbing(optimizer, chromosome: list[int], max_iterations: int = 20) -> list[int]:
    """Improve the GA result using deterministic local neighborhood search."""
    best_chromosome = chromosome[:]
    best_score = optimizer.calculate_fitness(best_chromosome).fitness
    context = optimizer.context

    for _ in range(max_iterations):
        improved = False

        for slot in range(1, GENES_PER_DAY - 1):
            can_swap = (
                best_chromosome[slot] in [Allele.SLEEP, Allele.NAP, Allele.REST]
                and best_chromosome[slot + 1] in [Allele.SLEEP, Allele.NAP, Allele.REST]
                and slot not in context.transit_slots
                and (slot + 1) not in context.transit_slots
                and best_chromosome[slot] != best_chromosome[slot + 1]
            )
            if not can_swap:
                continue
            candidate = best_chromosome[:]
            candidate[slot], candidate[slot + 1] = candidate[slot + 1], candidate[slot]
            candidate = optimizer.repair_chromosome(candidate)
            candidate_score = optimizer.calculate_fitness(candidate).fitness
            if candidate_score > best_score:
                best_chromosome = candidate
                best_score = candidate_score
                improved = True

        for slot in range(GENES_PER_DAY):
            if slot in context.mandatory_absolute:
                continue
            original_value = best_chromosome[slot]
            alternatives = [Allele.SLEEP, Allele.NAP, Allele.REST] if is_nap_allowed_slot(slot) else
[Allele.SLEEP, Allele.REST]
            for alternative in alternatives:
                if alternative == original_value:
                    continue
                candidate = best_chromosome[:]
                candidate[slot] = int(alternative)
                candidate = optimizer.repair_chromosome(candidate)
                candidate_score = optimizer.calculate_fitness(candidate).fitness
                if candidate_score > best_score:
                    best_chromosome = candidate
                    best_score = candidate_score
                    improved = True
                    break

        if not improved:
            break
    return best_chromosome
```

Cuplikan kode 4.7 Pemanggilan GA dan Hill Climbing dari weekly.py (src/sora/experiments/weekly.py)

```
best_ga, day_history = optimizer.run_genetic_algorithm_with_history(
    generations=200,
    hill_climbing_iterations=parameters.get("hill_climbing_iterations", 20),
)
for record in day_history:
    convergence_history.append({"day": day_name, **record})
best_schedule = run_hill_climbing(optimizer, best_ga)
result = optimizer.calculate_fitness(best_schedule)
```

Cuplikan kode 4.8 Endpoint optimasi mingguan (api.py)

```
@app.post("/api/weekly/optimize")
def optimize_weekly(request: WeeklyRequest) -> dict:
    schedule, metrics, history = run_weekly_optimization_with_history(
        resolve_dataset(request.dataset_name),
        parameters_from(request),
        request.target_name,
        random_seed=request.random_seed,
    )
    if schedule is None:
        raise HTTPException(status_code=404, detail="Subjek tidak ditemukan dalam dataset.")
    return {
        "target_name": request.target_name,
        "schedule": schedule,
        "observed_schedule": [metric.get("observed_schedule", [3] * 48) for metric in metrics],
        "metrics": metrics,
        "history": history,
    }
```

4.2 Hasil Kalibrasi Parameter

Kalibrasi parameter dilakukan terpisah dari eksperimen utama menggunakan grid search. Tiga parameter yang diuji ialah ukuran populasi 50 dan 100, laju mutasi 0,01; 0,05; dan 0,10, serta laju crossover 0,70; 0,80; dan 0,90. Kombinasi tersebut membentuk 18 konfigurasi. Setiap konfigurasi dinilai pada lima subjek dataset kalibrasi dan tiga hari representatif (Senin, Rabu, dan Sabtu). Nilai pada Tabel 4.2 merupakan rata-rata fitness kalibrasi untuk masing-masing kombinasi; skor yang lebih tinggi dipilih sebagai hasil terbaik menurut fungsi tujuan yang digunakan.

Tabel 4.2 Hasil Kalibrasi Parameter Menggunakan Grid Search

Ukuran Populasi	Laju Mutasi	Laju Crossover	Rata-rata Fitness
50	0,01	0,70	2,843
50	0,01	0,80	2,854
50	0,01	0,90	-263,881
50	0,05	0,70	2,904
50	0,05	0,80	2,904
50	0,05	0,90	-263,826
50	0,10	0,70	2,906
50	0,10	0,80	-263,824
50	0,10	0,90	2,915
100	0,01	0,70	-263,868
100	0,01	0,80	-263,851
100	0,01	0,90	2,884
100	0,05	0,70	2,905
100	0,05	0,80	-263,825
100	0,05	0,90	2,902
100	0,10	0,70	2,915
100	0,10	0,80	2,917
100	0,10	0,90	2,905

Sumber: Diolah penulis

Skor tertinggi pada kalibrasi adalah 2,917, yang diperoleh oleh ukuran populasi 100, mutasi 0,10, dan crossover 0,80. Nilai kalibrasi yang negatif pada beberapa kombinasi tetap mungkin terjadi karena fitness menjumlahkan manfaat tidur dengan penalti durasi, fragmentasi tidur, nap, dan regularitas. Nilai negatif bukan berarti data hilang; nilainya menunjukkan bahwa penalti pada fungsi tujuan lebih besar daripada komponen manfaat pada kasus yang diuji. Namun, hasil kalibrasi ini tidak digunakan sebagai konfigurasi eksperimen utama: metadata final mencatat parameter_source=default_fallback, ukuran populasi 50, mutasi 0,10, crossover 0,80, dan seed 42. Karena proses utama dijalankan dengan opsi skip-calibration, angka 2,917 harus dibaca sebagai hasil eksperimen kalibrasi terpisah, bukan hasil dari konfigurasi yang dipakai untuk menghasilkan 441 jadwal.

Grid search membentuk 18 kombinasi dari 2 ukuran populasi, 3 nilai mutasi, dan 3 nilai crossover. Pada setiap konfigurasi, lima subjek kalibrasi dinilai pada tiga hari representatif (Senin, Rabu, Sabtu); implementasi merata-ratakan skor hari dalam subjek terlebih dahulu, lalu merata-ratakan lima subjek. Nilai rata-rata 18 konfigurasi berada pada rentang -263,881 sampai 2,917. Enam konfigurasi memiliki rata-rata negatif sekitar -263,8, sedangkan dua belas konfigurasi lainnya memiliki rata-rata positif sekitar 2,84–2,92. Berkas hasil kalibrasi hanya menyimpan rata-rata per konfigurasi, bukan rincian fitness dan penalti per subjek-hari; karena itu penyebab spesifik skor negatif tiap konfigurasi tidak dapat dipastikan dari artefak final dan tidak diatribusikan ke satu aturan tertentu. Selain itu, kode menetapkan seed berbeda untuk indeks konfigurasi yang berbeda, sehingga selisih antarkonfigurasi juga dapat memuat variasi inisialisasi acak, bukan hanya pengaruh parameter.

4.3 Hasil Pengujian Optimasi Mingguan

Pengujian mingguan memakai satu baris kuesioner untuk setiap responden dan memproses hari Senin sampai Minggu secara berurutan. Seluruh 63 responden menghasilkan tujuh jadwal; 441 dari 441 hari berstatus optimized. Status tersebut berarti proses optimasi menghasilkan keluaran harian, bukan bukti bahwa setiap jadwal sudah ideal secara medis. Pada setiap hari, jadwal kerja dan kuliah serta slot transisinya dipertahankan sebagai kendala; sistem mengoptimasi aktivitas tidur pada slot lain. Kondisi akhir kelelahan, waktu mulai tidur utama, dan sebagian memori slot dari hari sebelumnya diteruskan ke hari berikutnya. GA dijalankan selama 200 generasi per hari dan pencarian lokal dijalankan sesuai alur implementasi pada Bab IV; seed harian menggunakan seed dasar 42 ditambah indeks hari. Ringkasan metrik dihitung dari keluaran harian eksperimen final. Tabel 4.3 merangkum keluaran, komponen fitness, dan ukuran sebarannya.

Tabel 4.3 Statistik Hasil Pengujian Optimasi Mingguan pada Seluruh Subjek Pengujian

Metrik	Nilai
Baris data / responden unik	63 / 63
Evaluasi harian	441 dari 441 berstatus optimized
Fitness hasil optimasi (rata-rata / median)	-2,2101 / 6,9072
Fitness hasil optimasi (minimum /	-1346,9761 / 7,7997

Metrik	Nilai
maksimum)	
Fitness baseline observasi (rata-rata)	-158,9597
Fitness delta (rata-rata)	156,7496
Hari fitness optimasi lebih tinggi dari baseline	399 dari 441 (90,48%)
Durasi tidur optimasi / baseline (jam per hari)	6,5045 / 6,5000
Perubahan slot tidur (rata-rata)	3,1474 slot (sekitar 1,57 jam)
Fatigue akhir (rata-rata)	0,6957
Fitness baseline observasi (median)	-212,9600
Fitness baseline observasi (rentang)	-1474,0858 sampai 7,5709
Fitness delta (median)	0,9759
Rentang fitness delta	-183,8144 sampai 1476,8317
Hari delta positif / sama / negatif	399 / 29 / 13 dari 441 hari
Durasi tidur optimasi (median; rentang)	6,5 jam; 3 sampai 10 jam
Recovery reward (rata-rata / median)	1,3766 / 1,4419
Recovery reward (rentang)	0,2400 sampai 2,3996
Penalti dinamis (rata-rata / median)	-2,0749 / 4,2000
Penalti dinamis (rentang)	-1350 sampai 4,5500
Penalti regularitas (rata-rata / median)	-2,7211 / 0
Penalti regularitas (rentang)	-400 sampai 0
Hari dengan 1 sesi nap / tanpa nap	298 / 143 dari 441 hari
Rentang fatigue akhir	0,0975 sampai 1,7865
Median perubahan slot tidur	2 slot
Rentang perubahan slot tidur	0 sampai 26 slot

Sumber: Diolah penulis

Rata-rata fitness hasil optimasi adalah -2,2101, sedangkan mediannya 6,9072. Perbedaan besar antara keduanya terjadi karena sebaran memiliki pencilan negatif: nilai minimum -1346,9761 dan maksimum 7,7997. Oleh sebab itu, rata-rata saja tidak mewakili hari yang paling umum. Nilai fitness baseline observasi juga memiliki rata-rata -158,9597 dan median -212,9600; rata-rata fitness delta sebesar 156,7496 perlu dibaca bersama median delta 0,9759 dan rentang -183,8144 sampai 1476,8317. Rata-rata delta yang tinggi terutama dipengaruhi beberapa baseline dengan penalti sangat besar, bukan berarti skor setiap hari meningkat sekitar 156 poin.

Secara berpasangan, fitness optimasi lebih tinggi daripada baseline pada 399 hari (90,48%), sama pada 29 hari, dan lebih rendah pada 13 hari. Jadi, hasil agregat mendukung perbaikan skor pada mayoritas hari, tetapi tidak menunjukkan bahwa sistem selalu memperbaiki setiap jadwal. Rata-rata durasi tidur hasil optimasi 6,5045 jam per hari, sementara baseline observasi 6,5000 jam. Selisih durasi rata-rata yang kecil menunjukkan bahwa perubahan skor tidak terutama berasal dari penambahan jumlah jam tidur; fungsi

tujuan juga memperhitungkan pemulihan, kepadatan aktivitas, penalti kualitas, dan regularitas.

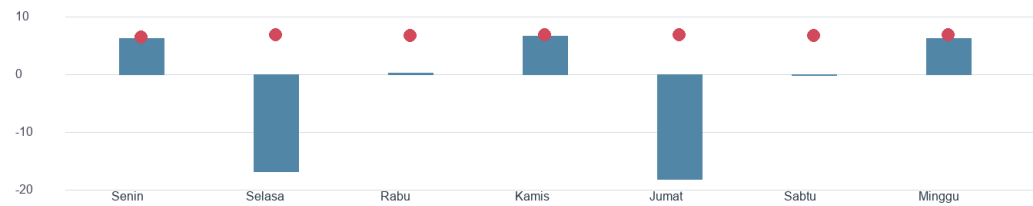
Rata-rata perubahan sebesar 3,1474 slot adalah jumlah slot tidur yang berbeda antara hasil optimasi dan jadwal observasi, dihitung dari gabungan slot yang ditambah atau dihilangkan. Karena satu slot setara 30 menit, besaran itu setara dengan 1,5737 jam-slot perubahan kumulatif, bukan berarti waktu mulai tidur bergeser tepat 1,57 jam. Median perubahan adalah 2 slot dan nilai maksimum 26 slot. Rata-rata fatigue akhir 0,6957 merupakan indeks internal model yang diwariskan antahari; nilainya berguna untuk membandingkan keluaran di dalam eksperimen ini, tetapi tidak ditafsirkan sebagai nilai klinis.

Rincian menurut hari menunjukkan bahwa median fitness optimasi relatif berdekatan, yaitu 6,5604 pada Senin; 6,9783 pada Selasa; 6,9236 pada Rabu; 6,9881 pada Kamis; 7,0150 pada Jumat; 6,8483 pada Sabtu; dan 6,9608 pada Minggu. Sebaliknya, rata-rata fitness per hari adalah 6,2715; -16,8166; 0,3628; 6,6801; -18,1428; -0,1931; dan 6,3675 secara berurutan. Perbedaan mean dan median terutama muncul karena enam keluaran memiliki fitness di bawah -100: dua pada Selasa, satu pada Rabu, dua pada Jumat, dan satu pada Sabtu. Dua kasus bahkan di bawah -500, masing-masing pada Selasa dan Jumat. Untuk konteks perubahan terhadap jadwal observasi, fitness optimasi lebih tinggi pada 58/63 hari Senin, 54/63 Selasa, 56/63 Rabu, 58/63 Kamis, 55/63 Jumat, 57/63 Sabtu, dan 61/63 Minggu; hari yang tersisa pada tiap kelompok terbagi menjadi skor sama atau lebih rendah. Dengan demikian hasil mingguan tidak seragam ant hari, tetapi mayoritas hari pada setiap hari dalam pekan tetap menunjukkan fitness yang lebih tinggi.

Rata-rata durasi tidur hasil optimasi per hari berkisar dari 6,4444 jam pada Kamis hingga 6,5397 jam pada Jumat; rata-rata fatigue akhir berkisar dari 0,5841 pada Minggu hingga 0,7734 pada Rabu. Rata-rata tujuh hari secara gabungan adalah 6,5045 jam dan 0,6957. Data baseline observasi mencatat 6,5 jam pada seluruh 441 hari, sehingga selisih rerata durasi kedua jadwal hanya 0,0045 jam per hari. Hasil ini menjelaskan bahwa kenaikan fitness tidak berasal dari peningkatan durasi rata-rata yang berarti, melainkan juga dari komponen pemulihan dan penalti di fungsi tujuan. Durasi tidur yang lebih tinggi atau fatigue internal yang lebih rendah tetap tidak membuktikan dampak kesehatan tanpa pengukuran aktual.

Fitness harian SORA dan hasil berpasangan terhadap observasi

A. Rerata dan median fitness (titik merah = median)



B. Pasangan fitness: optimasi lebih tinggi / sama / lebih rendah (n=63)



Gambar 4.2 Rerata, median, dan hasil berpasangan fitness menurut hari

Sumber: Diolah penulis dari 441 keluaran eksperimen mingguan

Keluaran komponen fitness memperlihatkan recovery reward rata-rata 1,3766 (median 1,4419; rentang 0,2400–2,3996). Penalti dinamis memiliki rata-rata –2,0749, median 4,2000, dan rentang –1350 sampai 4,5500; median positif kecil menunjukkan bahwa sebagian besar hasil memperoleh komponen ini positif, sedangkan nilai ekstrem negatif menurunkan rata-rata. Penalti regularitas bernilai nol pada 438 dari 441 hari dan mencapai –400 pada tiga hari; ini adalah penalti internal yang dibatasi model, bukan nilai SRI. Terdapat satu sesi nap pada 298 hari dan tidak ada sesi nap pada 143 hari. Tidak adanya nap tidak otomatis menandakan keluaran gagal karena nap merupakan komponen opsional dan fitness menilai seluruh jadwal. Durasi tidur hasil optimasi berkisar 3–10 jam per hari; nilai minimum dan maksimum tersebut merupakan keluaran model pada kasus individual, bukan rekomendasi durasi tidur bagi pengguna.

4.4 Hasil Demo Pipeline Skripsi

Pipeline demo skripsi dijalankan untuk mengemas keluaran eksperimen final menjadi artefak yang dapat diperiksa dan digunakan dalam penyusunan laporan. Berkas README.md menjelaskan isi paket, INSTRUCTIONS.md memuat petunjuk penggunaan, METADATA.json mencatat parameter dan seed, sedangkan REPORT.md dan REPORT.csv menyajikan ringkasan hasil. summary_report.csv merangkum keluaran mingguan; direktori results memuat ringkasan kalibrasi, perbandingan multi-subjek, metrik harian, dan jadwal mingguan. Artefak per responden mengandung data sensitif sehingga tidak ditampilkan dengan identitas pada naskah.

Metadata eksperimen utama mencatat seed 42, ukuran populasi 50, laju mutasi 0,10, laju crossover 0,80, serta sumber parameter default_fallback karena eksekusi utama menggunakan --skip-calibration. Ringkasan final memuat 63 responden dan 441 baris keluaran harian. Keterlacakan ini memungkinkan pembaca memeriksa konsistensi antara parameter, hasil agregat, dan berkas eksperimen tanpa menyamakan pipeline demo dengan kalibrasi parameter.

4.5 Perbandingan dengan Baseline

Perbandingan antarmetode dilakukan pada hari Rabu untuk 63 responden yang sama; seed setiap subjek merupakan seed dasar 42 ditambah indeks subjek. GA SORA dijalankan selama 150 generasi dan disempurnakan dengan pencarian lokal sebagaimana urutan implementasi pada modul baseline. Baseline greedy mengisi blok waktu bebas terpanjang menuju target tidur, sedangkan fixed schedule menempatkan pola tidur nominal pukul 22.00 selama tujuh jam. Ketiga jadwal dinilai dengan fungsi evaluasi yang sama setelah mekanisme repair diterapkan, sehingga angka pada Tabel 4.4 adalah metrik keluaran sesudah repair. Perbandingan ini merupakan uji hari Rabu, bukan perbandingan tujuh hari penuh.

Tabel 4.4 Statistik dan Hasil Berpasangan Metode pada Hari Rabu

Metrik	SORA	Greedy	Fixed
Rata-rata fitness	6,28	-52,81	-5630,96
Rata-rata fatigue akhir	0,74	1,18	0,89
Rata-rata durasi tidur (jam)	6,52	6,90	7,43
Fitness SORA lebih tinggi	—	59 dari 63	59 dari 63
Fitness tertinggi simultan	58 dari 63 data	—	—
Median fitness	6,59	-14,33	-13,56
Median fatigue akhir	0,72	1,18	0,74
Median durasi tidur (jam)	6,5	7,0	7,0
Fitness minimum	2,63	-1347,54	-75371,12
Fitness maksimum	6,88	6,86	6,86
Fatigue minimum	0,28	0,26	0,31
Fatigue maksimum	1,55	1,79	1,77
Durasi tidur minimum (jam)	6,5	6,5	2,5
Durasi tidur maksimum (jam)	7,0	7,0	18,0
Fitness SORA sama	—	4 dari 63	3 dari 63
Fitness SORA lebih rendah	—	0 dari 63	1 dari 63
Fatigue SORA lebih rendah	—	47 dari 63	38 dari 63
Fatigue SORA sama	—	4 dari 63	4 dari 63
Fatigue SORA lebih tinggi	—	12 dari 63	21 dari 63
Durasi tidur SORA lebih pendek	—	49 dari 63	42 dari 63
Durasi tidur SORA sama	—	14 dari 63	13 dari 63
Durasi tidur SORA lebih panjang	—	0 dari 63	8 dari 63

Sumber: Diolah penulis

Pada 63 pasangan data hari Rabu, rata-rata fitness SORA adalah 6,2767, dibandingkan -52,8108 untuk greedy dan -5630,9624 untuk fixed. Median masing-masing adalah 6,5881; -14,3300; dan -13,5598. Nilai minimum-maksimum memperlihatkan pengaruh pencilan: SORA 2,6283–6,8822, greedy -1347,5356–6,8552, dan fixed -75371,1200–6,8552. Secara berpasangan, fitness SORA lebih tinggi daripada greedy pada 59 subjek, sama pada 4, dan tidak pernah lebih rendah; dibanding fixed, SORA lebih tinggi pada 59, sama pada 3, dan lebih rendah pada 1 subjek. SORA menjadi metode dengan fitness tertinggi terhadap kedua baseline sekaligus pada 58 dari 63 subjek. Jadi, temuan mendukung keunggulan fitness SORA pada mayoritas sampel, tetapi menyatakan kemenangan mutlak pada setiap orang akan keliru.

Rata-rata fatigue SORA (0,7354) lebih rendah daripada greedy (1,1828) dan fixed (0,8908); mediannya masing-masing 0,7195; 1,1800; dan 0,7415. Pada perbandingan

berpasangan, fatigue SORA lebih rendah pada 47 subjek dan sama pada 4 subjek dibanding greedy; pada 12 subjek fatigue SORA lebih tinggi. Dibanding fixed, fatigue SORA lebih rendah pada 38 subjek, sama pada 4, dan lebih tinggi pada 21 subjek. Rentang fatigue adalah 0,2824–1,5500 untuk SORA, 0,2650–1,7900 untuk greedy, dan 0,3107–1,7700 untuk fixed. Arah rata-rata, median, dan mayoritas pasangan selaras: nilai fatigue SORA lebih rendah secara agregat dan pada mayoritas pasangan, meski tidak pada semua subjek.

Rata-rata durasi tidur SORA 6,5159 jam (median 6,5), dibandingkan greedy 6,9048 jam (median 7,0) dan fixed 7,4286 jam (median 7,0). SORA memiliki durasi lebih pendek daripada greedy pada 49 subjek, sama pada 14, dan tidak lebih panjang pada pasangan mana pun. Dibanding fixed, durasi SORA lebih pendek pada 42 subjek, sama pada 13, dan lebih panjang pada 8. Rentang durasi masing-masing adalah 6,5–7,0; 6,5–7,0; dan 2,5–18,0 jam. Baseline fixed ditetapkan nominal tujuh jam sebelum repair, tetapi proses repair dapat memperluas blok tidur dan menjembatani celah pendek, sehingga keluaran aktualnya tidak selalu tujuh jam. Perbandingan durasi ini karena itu menggambarkan jadwal sesudah repair, bukan baseline tujuh jam yang dipertahankan tanpa perubahan.

Ketiga jadwal dinilai dengan fungsi fitness yang sama; angka fitness adalah skor komposit tanpa satuan, bukan durasi atau tingkat kesalahan. Rata-rata fixed (–5630,9624) jauh di bawah mediannya (–13,5598), dan nilai minimumnya –75371,1200, sehingga beberapa pencilan mendominasi rerata. Komponen pencilan minimum dapat ditelusuri: sesudah repair, jadwal fixed tersebut berisi 36 slot tidur atau 18 jam, sedangkan target observasinya 13 slot atau 6,5 jam. Tabel 4.5 memisahkan perhitungan penalti serta komponen skor tersebut dari uraian agar setiap ekspresi matematis terbaca sebagai baris tersendiri. Skor ekstrem berasal dari keluaran repair yang membengkak dan bobot penalti durasi, bukan kesalahan pembacaan satuan. Perbedaan mean–median itu menjelaskan mengapa median, rentang, dan hitungan berpasangan perlu dibaca bersama mean. Setelah repair, baseline fixed dapat berubah dari target nominal tujuh jam; batas ini membatasi interpretasi keunggulan antarmetode.

Tabel 4.5 Rincian Perhitungan Komponen Fitness pada Kasus Ekstrem

$-200 \times (36 - 18)^2 = -64.800$
$-20 \times (36 - 13)^2 = -10.580$
$12 \times 0,35 = 4,20$
$-64.800 - 10.580 + 4,20 = -75.375,80$
$-75.375,80 + 4,68 = -75.371,12$
$-150 \times 3^2 = -1.350$

Keterangan: baris pertama sampai kelima menjabarkan skor minimum metode fixed pada hari Rabu; baris terakhir menunjukkan penalti survival mode pada keluaran mingguan hari Jumat.

4.6 Pembahasan Hasil

4.6.1 Analisis Ringkas Per Subjek

Sebagai ilustrasi anonim, satu kasus dengan rerata fitness mingguan tepat pada median responden dipilih untuk ditelusuri. Pada hari Rabu, fitness SORA sebesar 7,2948, sedangkan jadwal observasi memperoleh 7,0043; selisihnya 0,2906. Keduanya memiliki durasi tidur utama 6,5 jam. Jadwal hasil optimasi menambahkan tiga slot nap pada rentang

tengah hari, sekitar pukul 12.00–13.30. Contoh ini memperlihatkan bahwa skor dapat berubah tanpa perubahan durasi tidur utama, tetapi satu kasus tidak mewakili seluruh responden dan tidak membuktikan dampak tidur aktual.

Pada eksperimen mingguan, seluruh 63 responden memperoleh keluaran tujuh hari, tetapi kata “berhasil” pada status optimized hanya menunjukkan bahwa optimasi harian selesai menghasilkan jadwal. Rata-rata fitness $-2,2101$ berbeda jauh dari median $6,9072$ karena beberapa kasus memperoleh penalti besar. Salah satu nilai terendah adalah $-1346,9761$ pada hari Jumat; komponen penalti dinamisnya -1350 dan jumlah sesi nap nol. Pada hari dengan jadwal wajib yang sangat padat, sistem masuk survival mode. Aturan implementasi memberi penalti saat tiga sesi nap tidak terbentuk; nilai komponen tersebut dituliskan terpisah pada Tabel 4.5. Jadi pencilan ini dapat ditelusuri ke aturan eksplisit fungsi fitness, bukan salah salin angka. Namun, besarnya penalti juga menunjukkan bahwa bobot aturan tersebut sangat dominan dan perlu dievaluasi ulang.

Pada perbandingan hari Rabu, SORA memperoleh fitness lebih tinggi daripada greedy pada 59 subjek dan seri pada 4; terhadap fixed, SORA lebih tinggi pada 59, seri pada 3, dan lebih rendah pada 1. Fitness SORA juga menjadi yang tertinggi dari ketiga metode pada 58 subjek. Untuk fatigue, SORA lebih rendah pada 47 pasangan terhadap greedy dan 38 terhadap fixed; angka tersebut tidak berarti semua pengguna memperoleh penurunan fatigue. Ketiga metrik memberi gambaran berbeda: fitness adalah fungsi tujuan komposit, fatigue adalah indeks internal model, sedangkan durasi hanya menghitung jumlah slot tidur. Hasil yang baik pada satu metrik tidak otomatis berarti nilai terbaik pada metrik lain.

Hasil yang dibahas merupakan evaluasi komputasional dengan parameter eksperimen yang tercatat, bukan validasi klinis atau pengukuran langsung kualitas tidur. Penalti regularitas internal bukan perhitungan Sleep Regularity Index (SRI) standar. Sebagai contoh, delta fitness terendah $-183,8144$ berkaitan dengan penalti regularitas -400 , yaitu batas penalti yang diterapkan ketika waktu mulai tidur berbeda jauh dari jangkar hari sebelumnya. Mekanisme ini menjelaskan penurunan skor tersebut, tetapi belum membuktikan bahwa pola tidur secara biologis tidak teratur. SORA juga tidak menghasilkan diagnosis medis atau bukti sebab-akibat terhadap produktivitas.

Validitas generalisasi masih dibatasi oleh 63 responden, horizon pengujian tujuh hari, satu konfigurasi parameter utama, dan satu seed acak. Kalibrasi dilakukan pada lima subjek terpisah, sedangkan konfigurasi kalibrasi terbaik tidak dipakai pada eksperimen utama dan seed antarkonfigurasi kalibrasi berbeda. Perbandingan baseline hanya menguji hari Rabu dan baseline fixed berubah setelah repair. Analisis yang dilaporkan bersifat deskriptif; berkas hasil final tidak menyertakan uji inferensial atau interval kepercayaan, sehingga perbedaan angka tidak boleh disebut signifikan secara statistik. Batas-batas tersebut harus diperhitungkan saat membaca hasil dan menjadi sasaran perbaikan metodologis sebelum menyimpulkan kinerja pada populasi atau kondisi lain.

4.7 Ringkasan Pembahasan

Secara umum, hasil final menunjukkan bahwa SORA dapat memproses data kuesioner, menghasilkan jadwal untuk tujuh hari, membandingkan keluaran dengan baseline, dan mengemas ringkasan eksperimen. Pada 399 dari 441 hari fitness hasil optimasi lebih tinggi daripada jadwal observasi, sedangkan pada perbandingan Rabu fitness SORA

tertinggi dibanding kedua baseline secara simultan pada 58 dari 63 responden. Temuan ini merupakan perbandingan skor komputasional dalam konfigurasi penelitian; temuan tersebut tidak membuktikan peningkatan kualitas tidur aktual atau dampak klinis.

4.8 Keterbatasan Pengujian

Hasil pengujian dibatasi pada 63 responden dan horizon tujuh hari, dengan satu konfigurasi utama serta satu seed acak. Analisis yang tersedia bersifat deskriptif dan tidak menyertakan uji inferensial atau interval kepercayaan. Perbandingan antarmetode hanya dilakukan pada hari Rabu; selain itu, proses repair dapat mengubah jadwal fixed nominal tujuh jam. Implementasi menggunakan regularity penalty internal dan belum menghitung SRI standar, serta belum memvalidasi jadwal terhadap pengukuran tidur aktual. Oleh karena itu, fitness dan fatigue dibahas sebagai keluaran internal sistem, bukan ukuran klinis atau bukti manfaat kesehatan.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan implementasi dan eksperimen pada data final, kesimpulan penelitian disusun untuk menjawab tujuan optimasi jadwal serta membatasi klaim pada bukti yang diukur.

SORA berhasil menghasilkan jadwal tidur komputasional tujuh hari untuk 63 responden unik dari 63 baris data. Seluruh 441 evaluasi harian berstatus optimized dengan seed 42 dan parameter eksperimen utama populasi 50, mutasi 0,10, serta crossover 0,80. Keberhasilan ini menunjukkan bahwa alur aplikasi dapat memproses data dan menghasilkan keluaran untuk setiap hari; status tersebut tidak dengan sendirinya menyatakan bahwa jadwal merupakan rekomendasi klinis.

Pada optimasi mingguan, rata-rata fitness hasil optimasi adalah $-2,2101$ dengan median $6,9072$; rata-rata durasi tidur $6,5045$ jam. Fitness optimasi melebihi baseline observasi pada 399 hari (90,48%), sama pada 29 hari, dan lebih rendah pada 13 hari. Median fitness delta $0,9759$ lebih mewakili perubahan tipikal daripada rerata delta $156,7496$ yang dipengaruhi pencilan. Temuan ini menjawab tujuan pengukuran kualitas relatif jadwal berdasarkan fungsi fitness, bukan akurasi prediksi.

Pada perbandingan hari Rabu untuk 63 responden, rata-rata fitness SORA $6,2767$ dan median $6,5881$ lebih tinggi daripada greedy (rata-rata $-52,8108$; median $-14,3300$) maupun fixed (rata-rata $-5630,9624$; median $-13,5598$). SORA memiliki fitness lebih tinggi daripada setiap baseline masing-masing pada 59 dari 63 responden, dan menjadi yang tertinggi dibanding keduanya sekaligus pada 58 responden. Kesimpulan ini berlaku untuk konfigurasi, data, hari, dan fungsi evaluasi yang diuji; tidak berarti SORA unggul pada seluruh metrik atau setiap responden.

Grid search terpisah pada lima subjek menghasilkan skor rata-rata tertinggi $2,917$ pada populasi 100, mutasi 0,10, dan crossover 0,80. Namun, metadata eksperimen utama mencatat konfigurasi fallback populasi 50, mutasi 0,10, dan crossover 0,80 karena pengujian utama melewati kalibrasi. Oleh sebab itu, hasil mingguan dan baseline dalam skripsi ini tidak boleh diatribusikan ke konfigurasi pemenang grid search.

Secara keseluruhan, eksperimen menunjukkan bahwa SORA mampu menghasilkan dan membandingkan jadwal menggunakan fitness komposit. Penelitian ini belum menghitung SRI standar secara penuh, belum menguji dampak jadwal pada kualitas tidur nyata atau produktivitas, dan tidak memberikan diagnosis. Klaim kesimpulan dibatasi pada keluaran komputasional dari dataset dan konfigurasi yang tersedia.

5.2 Keterbatasan Penelitian

Penelitian ini menggunakan 63 responden unik dan membatasi optimasi pada tujuh hari. Eksperimen utama menggunakan satu seed dan parameter default/fallback karena kalibrasi dilewati; baseline dibandingkan hanya pada hari Rabu. Tidak tersedia uji inferensial atau interval kepercayaan, pengukuran langsung atas kualitas tidur, maupun perhitungan SRI standar. Beberapa metrik seperti fitness, fatigue, dan regularity penalty merupakan ukuran internal fungsi evaluasi, sehingga hasil tidak dapat digeneralisasi sebagai rekomendasi medis.

5.3 Saran

Berdasarkan hasil dan batas penelitian yang telah dijelaskan, pengembangan selanjutnya disarankan sebagai berikut.

Periksa kembali bobot penalti pada kondisi survival mode, terutama penalti 1350 ketika sesi nap tidak terbentuk. Uji beberapa skenario jadwal padat dan laporkan komponen penalti secara terpisah agar perbaikan fitness tidak hanya didorong oleh perubahan satu aturan yang sangat dominan.

Ulangi eksperimen dengan beberapa seed acak dan laporkan rata-rata, median, simpangan baku, rentang, serta hasil berpasangan. Langkah ini diperlukan untuk menilai apakah hasil yang dilaporkan stabil terhadap variasi inisialisasi Genetic Algorithm.

Bandingkan SORA dengan metode optimasi tambahan menggunakan data, kendala, fungsi fitness, seed, dan anggaran komputasi yang sama. Baseline fixed perlu dievaluasi konsistensinya: jika maksud baseline adalah tepat tujuh jam mulai pukul 22.00, tetapkan aturan agar proses repair tidak mengubah durasi tersebut, atau nyatakan secara eksplisit perubahan setelah repair.

Perluas validasi dengan data longitudinal yang lebih beragam dan, bila tersedia, data wearable yang telah mendapat persetujuan etik dan persetujuan responden. Data aktual tersebut dapat digunakan untuk menguji kesesuaian prediksi jadwal dengan perilaku tidur, bukan hanya skor simulasi.

Jika regularitas tidur menjadi sasaran evaluasi, hitung SRI sesuai definisi dan kebutuhan data aslinya, lalu laporkan terpisah dari penalti regularitas internal SORA. Hal ini mencegah dua ukuran yang berbeda diberi nama atau makna yang sama.

Laporkan artefak eksperimen (metadata parameter, seed, data yang telah dianonimkan, ringkasan metrik, dan skrip yang diperlukan) secara konsisten agar pembaca dapat menelusuri asal setiap angka dan mengulang analisis tanpa membuka identitas responden.

5.4 Penutup

Penelitian ini menghasilkan prototipe optimasi jadwal tidur dan artefak eksperimen yang dapat ditelusuri. Kesimpulan dibatasi pada data dan konfigurasi yang diuji; validasi lebih luas dan pengukuran aktual tetap diperlukan sebelum hasil digunakan untuk menyatakan manfaat terhadap tidur atau kesehatan.